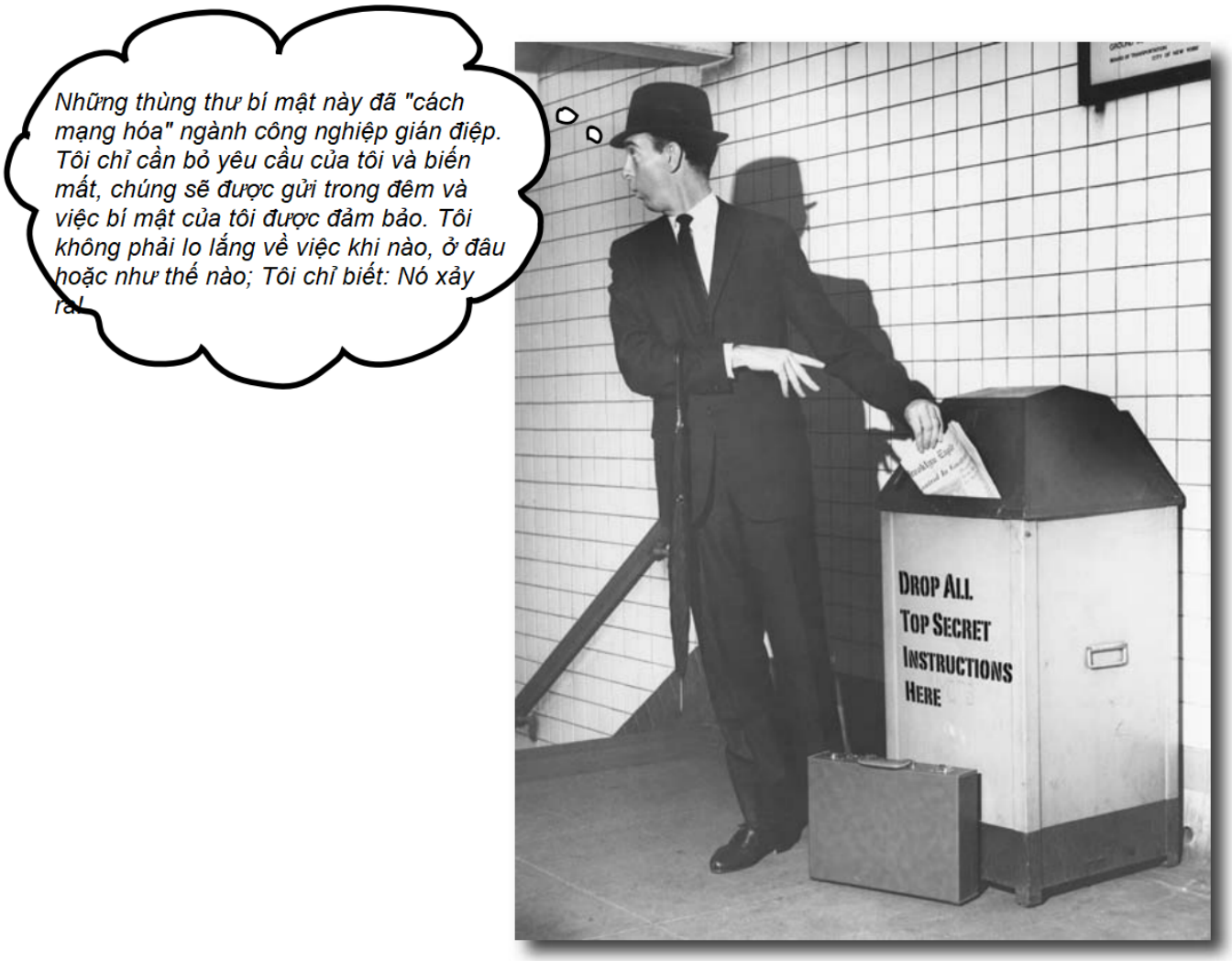
Chương 6: Command Pattern – Đóng gói yêu cầu

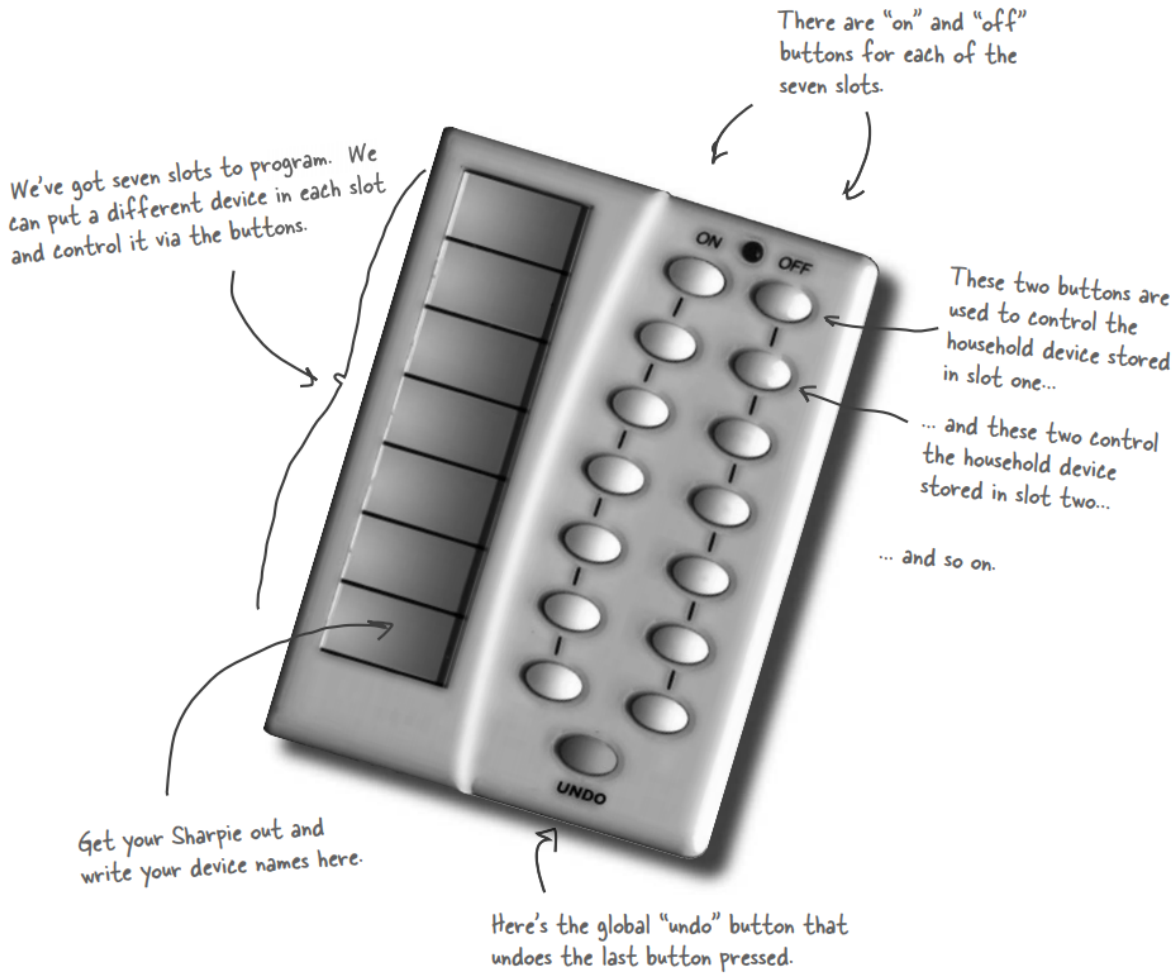


Command Pattern – Đóng gói yêu cầu

Trong chương này, chúng tôi đưa việc đóng gói lên một cấp độ hoàn toàn mới: đóng gói lời gọi (command) phương thức. Đúng vậy, bằng cách đóng gói lời gọi phương thức, Command pattern có thể kết tinh các phần xử lý để đối tượng gọi xử lý không cần phải lo lắng về cách thực hiện, nó chỉ sử dụng phương thức của chúng ta để hoàn thành nó. Với Command pattern, chúng ta cũng có thể thực hiện một số điều với các cách gọi phương thức được đóng gói này, như lưu chúng để ghi log hoặc tái sử dụng chúng để thực hiện undo trong code của chúng ta.

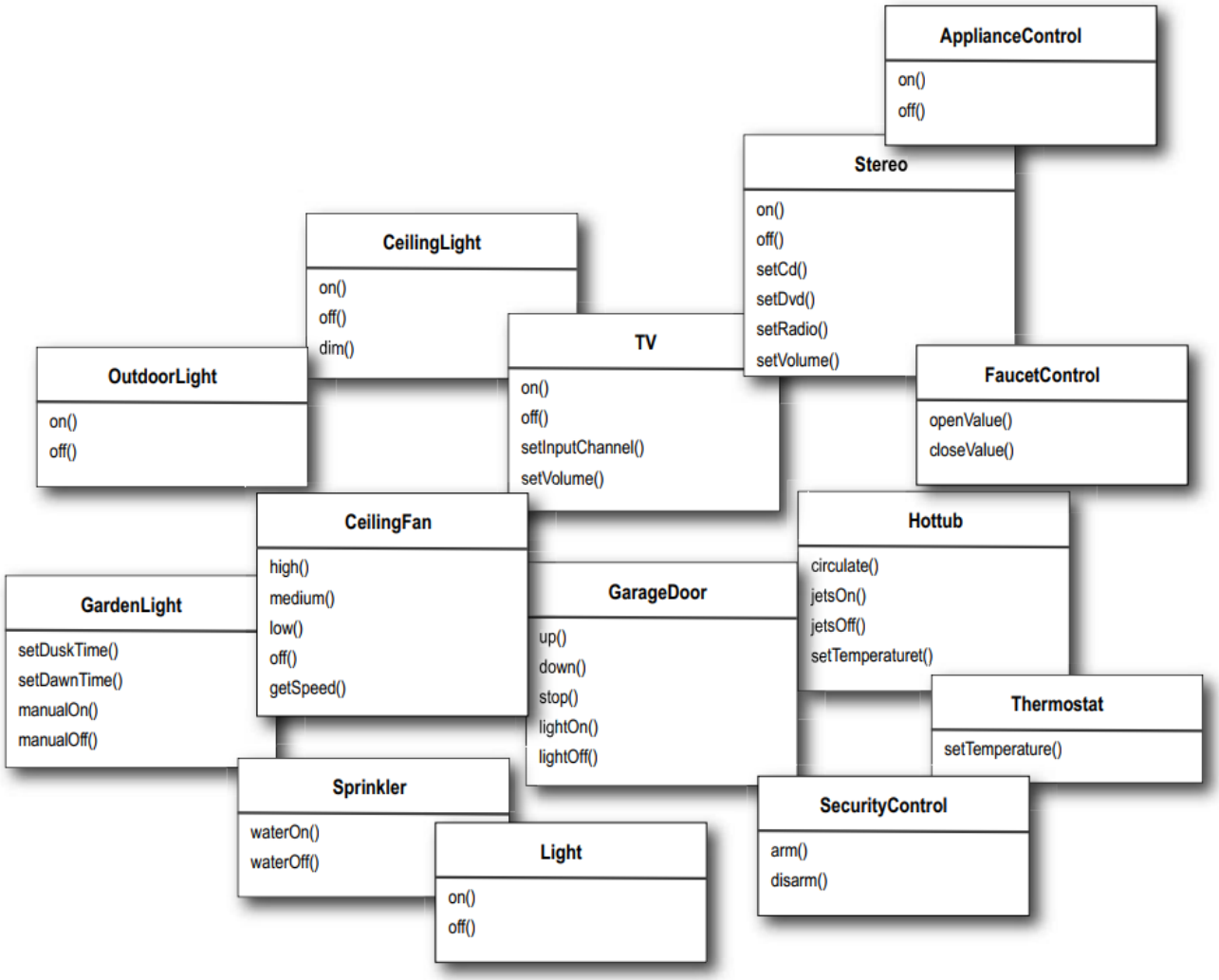
Chào các bạn!
Gần đây tôi đã nhận được một bản demo và tóm tắt từ Johnny Hurricane, CEO của Weather-O-Rama, trên trạm thời tiết có thể mở rộng mới của họ. Tôi phải nói rằng, tôi rất ấn tượng với kiến trúc phần mềm và tôi đã muốn yêu cầu bạn thiết kế API cho Điều khiển từ xa tự động hóa trong gia đình (Home Automation Remote Control) mới của chúng tôi. Đổi lại cho các dịch vụ của bạn, chúng tôi rất vui khi được thưởng cho bạn các tùy chọn cổ phiếu trong Home Automatic hoặc Bust, Inc.
Tôi đã bao gồm một bản gốc của điều khiển từ xa hiện có của chúng tôi cho sự chú ý của bạn. Điều khiển từ xa có bảy khe lập trình (mỗi khe có thể được gán cho một thiết bị gia dụng khác nhau) cùng với các nút bật/tắt tương ứng cho từng khe (slot). Điều khiển từ xa cũng có nút undo chung cho cả remote.
Tôi cũng kèm theo một tập hợp các lớp Java trên đĩa CD-R được tạo bởi các nhà cung cấp khác nhau để điều khiển các thiết bị tự động hóa gia đình như đèn, quạt, bồn nước nóng, thiết bị âm thanh và các thiết bị điều khiển tương tự khác.
Chúng tôi muốn bạn tạo một API để lập trình từ xa để mỗi khe có thể được chỉ định để điều khiển một thiết bị hoặc bộ thiết bị. Lưu ý rằng điều quan trọng là chúng tôi có thể kiểm soát các thiết bị hiện tại trên đĩa và bất kỳ thiết bị nào trong tương lai mà các nhà cung cấp có thể cung cấp.
Với công việc bạn đã làm trên trạm thời tiết Weather-O-Rama, (https://toihocdesignpattern.com/head-first-design-patterns-tieng-viet-chuong-2-observer-pattern.html) chúng tôi biết bạn sẽ làm rất tốt công việc điều khiển từ xa của chúng tôi!
Chúng tôi mong muốn được nhìn thấy thiết kế của bạn.
Trân trọng,
Bill (X-10) Thompson, Giám đốc điều hành.

Đã có phần cứng miễn phí! Hãy cùng kiểm tra điều khiển từ xa...



Nhìn vào các lớp được cung cấp

Kiểm tra các lớp từ nhà cung cấp trên đĩa CD-R. Chúng sẽ cung cấp cho bạn một số ý tưởng về giao diện của các đối tượng chúng ta cần điều khiển từ xa.



Có vẻ như chúng ta có khá nhiều lớp ở đây, và không dễ để đưa ra một bộ interface chung. Không chỉ vậy, trong tương lai, có vẻ như chúng ta có thể có các lớp nhiều hơn. Thiết kế API điều khiển từ xa sẽ rất thú vị.

Hội thoại

Đồng đội của bạn đã thảo luận về cách thiết kế API điều khiển từ xa …

Sue: Vâng, chúng ta đã có một thiết kế khác để làm. Quan sát đầu tiên của tôi là chúng ta đã có một điều khiển từ xa đơn giản với các nút on và off nhưng với một tập hợp các lớp từ nhà cung cấp khá đa dạng.

Mary: Vâng, tôi nghĩ rằng chúng ta đã thấy một loạt các lớp với các phương thức **on()** và **off()** nhưng ở đây chúng ta cũng có các phương thức như **dim()**, **setTemperature()**, **setVolume()**, **setDirection()**.

Sue: Không chỉ vậy, có vẻ như chúng ta có thể có các lớp từ nhà cung cấp nhiều hơn trong tương lai với các phương thức đa dạng hơn.

Mary: Tôi nghĩ điều quan trọng là chúng ta nên xem điều này như một sự tách biệt các mối quan tâm: điều khiển từ xa nên biết cách giải thích các lần nhấn nút và đưa ra yêu cầu, nhưng nó không nên biết nhiều về hành động khi nhấn nút đó sẽ làm gì (ví dụ điều khiển từ xa không cần phải biết cách bật bồn nước nóng).

Sue: Nghe có vẻ là thiết kế tốt. Nhưng nếu điều khiển từ xa chỉ biết cách thực hiện các yêu cầu chung chung, làm thế nào để chúng ta thiết kế điều khiển từ xa để nó có thể gọi một hành động, ví dụ, bật đèn hoặc mở cửa nhà để xe?

Mary: Tôi không chắc chắn, nhưng chúng ta không muốn điều khiển từ xa phải biết chi tiết cụ thể của các lớp từ nhà cung cấp.

Sue: Ý bạn là gì?

Mary: Chúng ta không muốn điều khiển từ xa bao gồm một tập hợp các câu lệnh if, giống như:
if slot1 == Light, then light.on(), else if slot1 = Hottub then hottub.jetsOn()
Chúng ta biết rằng đó là một thiết kế xấu.

Sue: Tôi đồng ý. Bất cứ khi nào một lớp từ nhà cung cấp mới xuất hiện, chúng ta sẽ phải sửa đổi code, có khả năng tạo ra lỗi và nhiều công việc hơn cho chính chúng ta!

Một người khác – Joe: Này, tôi không thể nghe lỏm được. Kể từ Chương 1 (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html), tôi đã tập trung vào các mẫu thiết kế. Có một mẫu hình được gọi là "Command Pattern" mà tôi nghĩ có thể giúp được.

Mary: Vâng? Hãy cho chúng tôi biết thêm đi.

Joe: Command Pattern cho phép bạn tách rời đối tượng yêu cầu hành động khỏi đối tượng thực sự thực hiện hành động. Vì vậy, ở đây đối tượng yêu cầu sẽ là remote và đối tượng thực hiện hành động sẽ là một instance của một trong các lớp từ nhà cung cấp của bạn.

Sue: Làm thế nào có thể? Làm thế nào chúng ta có thể tách chúng ra? Rốt cuộc, khi tôi nhấn một nút, điều khiển từ xa vẫn phải bật đèn.

Joe: Bạn có thể làm điều đó bằng cách đưa những "command object" vào thiết kế của bạn. Một command object đóng gói một yêu cầu (request) để làm một cái gì đó (như bật đèn) lên một đối tượng cụ thể (giả sử, đối tượng Đèn phòng khách). Vì vậy, nếu chúng ta lưu trữ một command object cho mỗi nút,

khi nhấn nút, chúng ta yêu cầu command object thực hiện một số công việc. Remote không biết gì về công việc, nó chỉ chứa một command object biết cách nói chuyện với đúng đối tượng để hoàn thành công việc. Vì vậy, bạn sẽ thấy, object Remote được tách biệt ra với các object Light!

Sue: Điều này nghe có vẻ như đã đi đúng hướng.

Mary: Tuy nhiên, tôi đã có một thời gian khó khăn để nghĩ về mẫu này.

Joe: Cho rằng các vật thể rất tách rời nhau, nó hơi khó hình dung mẫu thực sự hoạt động như thế nào.

Mary: Hãy để tôi xem nếu tôi ít nhất có ý tưởng đúng: sử dụng mẫu này, chúng ta có thể tạo API trong đó các command object có thể được đưa vào các khe slots, cho phép các đoạn code của remote đơn giản hơn. Và, các command object gói gọn cách thực hiện nhiệm vụ "tự động hóa nhà" thành một object cần thực hiện.

Joe: Vâng, tôi nghĩ vậy. Tôi cũng nghĩ rằng mẫu này có thể giúp bạn với nút Undo, nhưng tôi chưa nghiên cứu phần đó.

Mary: Điều này nghe có vẻ rất đáng khích lệ, nhưng tôi nghĩ rằng tôi có một chút việc phải làm để thực sự "có" được mẫu.

Sue: Tôi cũng vậy.

Trong khi đó, trở lại Diner…, hoặc là, Giới thiệu ngắn gọn về Command Pattern

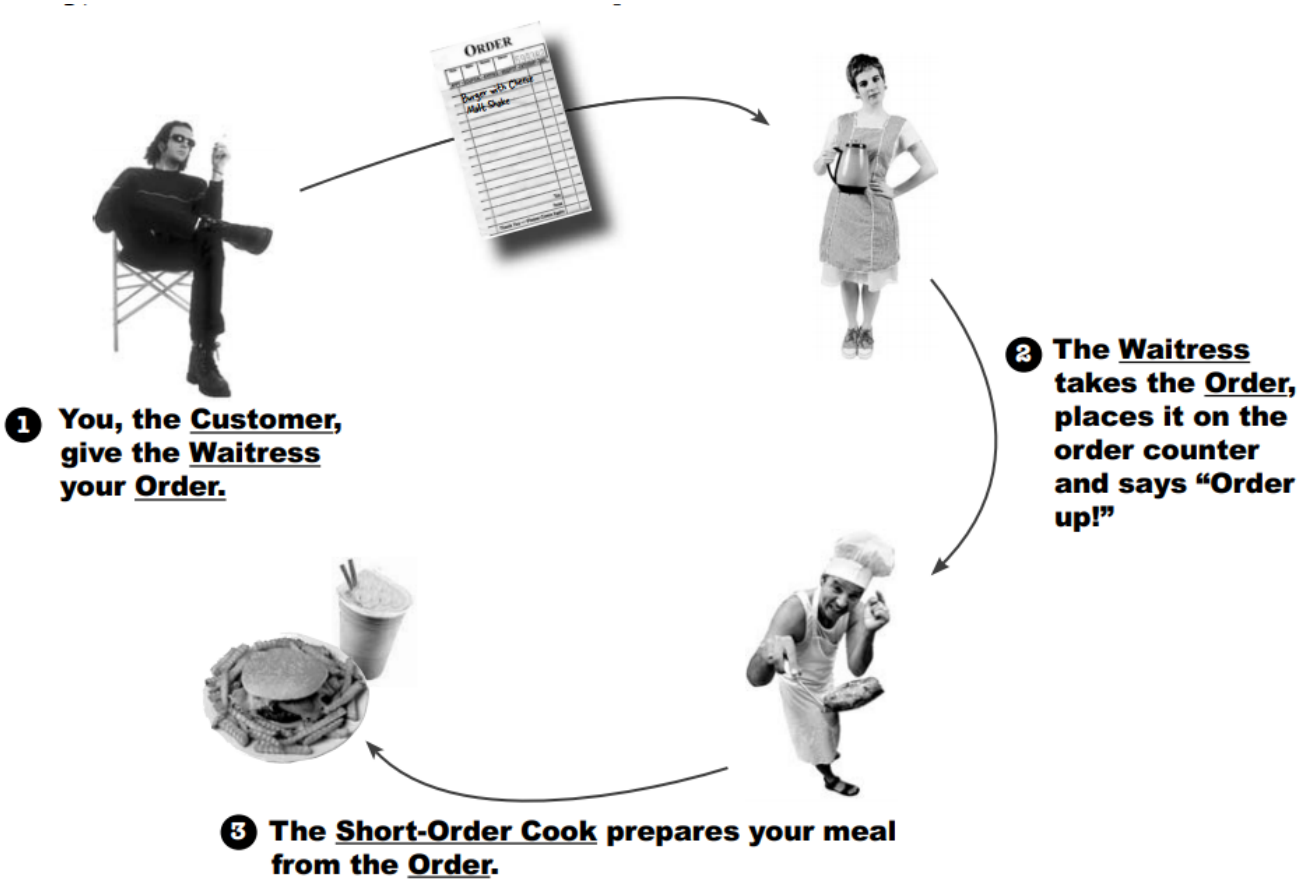


Như Joe đã nói, hơi khó để hiểu Command Pattern bằng cách chỉ nghe mô tả của nó. Nhưng đừng sợ, chúng tôi có vài người bạn sẵn sàng giúp đỡ: hãy nhớ đến quán ăn thân thiện của chúng tôi từ Chương 1 (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html)? Đã được một lúc kể từ khi chúng ta đến thăm Alice, Flo và người đầu bếp, nhưng chúng tôi có lý do chính đáng để quay trở lại (tốt, ngoài thức ăn và cuộc trò chuyện tuyệt vời): thực khách sẽ giúp chúng tôi hiểu hơn về mẫu Command.

Vì vậy, hãy đi một đường vòng ngắn trở lại quán ăn và nghiên cứu các tương tác giữa khách hàng (customer), nhân viên phục vụ (waitress), đơn đặt hàng (order) và đầu bếp (cook). Thông qua các tương tác này, bạn sẽ hiểu được các đối tượng liên quan đến Command Pattern và cũng có cảm giác về cách thức tách rời hoạt động. Sau đó, chúng ta sẽ loại bỏ API cũ của remote.

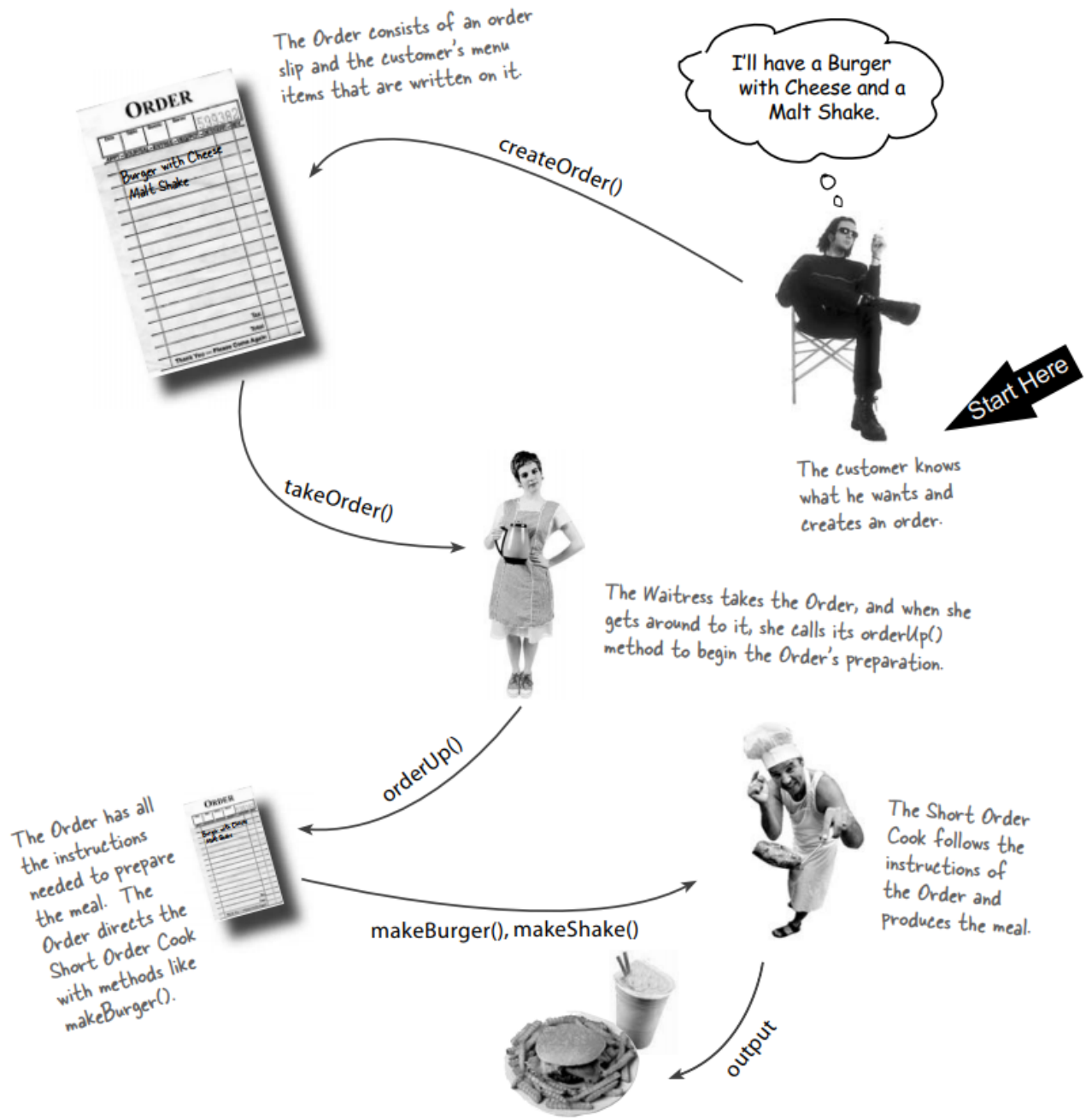
Checking in tại Objectville Diner …

Được rồi, tất cả chúng ta đều biết cách thức hoạt động của Diner



Hãy nghiên cứu sự tương tác để hiểu Command Pattern…

… Và đưa ra bữa tối này là ở Objectville, hãy nghĩ về các cuộc gọi đến Object và Method liên quan!



Vai trò và trách nhiệm của Objectville Diner

Một Order đóng gói một yêu cầu để chuẩn bị một bữa ăn.

Hãy nghĩ về Order như một đối tượng, một đối tượng đại diện yêu cầu để chuẩn bị một bữa ăn. Giống như bất kỳ đối tượng nào, nó có thể được truyền đi các class khác – từ Waitress đến quầy đặt hàng hoặc đến người Waitress tiếp theo đảm nhận ca làm việc. Order object có một interface chỉ bao gồm một phương thức **orderUp()**, gói gọn các hành động cần thiết để chuẩn bị bữa ăn. Nó cũng có một tham chiếu đến đối tượng cần chuẩn bị nó (trong trường hợp của chúng tôi là đầu bếp, Cook). Nó được gói gọn trong đó Waitress không phải biết những gì cần làm trong order hay ngay cả những người đầu bếp; Waitress chỉ cần cầm order, đưa qua cửa sổ nhà bếp và gọi “Order up!”

Công việc của Waitress là lấy Order và gọi phương thức orderUp()

Công việc của Waitress rất dễ dàng: nhận order từ khách hàng, sau đó đi đến quầy đặt hàng, và gọi phương thức **orderUp()** để chuẩn bị bữa ăn. Như chúng ta đã thảo luận, ở Objectville, Waitress thực sự không phải là người lo lắng về những gì mà trên order ghi hoặc đầu bếp nào sẽ nấu; cô ấy chỉ biết **Order** có một phương thức **orderUp()** mà cô ấy có thể gọi để hoàn thành công việc.

Bây giờ, trong suốt cả ngày, phương thức **TakeOrder()** của Waitress được tham số hóa với các order khác nhau từ các khách hàng khác nhau, nhưng điều đó không làm cô ấy bối rối; cô ấy biết tất cả các order đều hỗ trợ phương thức **orderUp()** và cô ấy có thể gọi **orderUp()** bất cứ khi nào cô ấy cần một bữa ăn được chuẩn bị.

Đầu bếp (Short-Order Cook) có kiến thức cần thiết để chuẩn bị bữa ăn.

Short-Order Cook là đối tượng thực sự biết cách chuẩn bị bữa ăn. Khi Waitress đã gọi phương thức **orderUp();** Short-Order Cook tiếp quản và thực hiện tất cả các method cần thiết để tạo ra bữa ăn.

Lưu ý Waitress và đầu bếp hoàn toàn tách rời: Waitress có Order gói gọn các chi tiết của bữa ăn; cô ấy chỉ cần gọi một phương thức trên mỗi Order để chuẩn bị.

Tương tự như vậy, Short-Order Cook nhận được danh sách các món ăn từ Order; anh ta không bao giờ cần phải liên lạc trực tiếp với Waitress.

Kiên nhẫn, chúng ta đã đến đó…

Hãy nghĩ về Diner như một mô hình cho mẫu thiết kế OO, cho phép chúng ta tách một đối tượng □"yêu cầu" (request) hành động khỏi các đối tượng receiver sẽ thực hiện các yêu cầu đó. Ví dụ, trong API điều khiển từ xa của chúng ta, chúng ta cần tách code được gọi khi nhấn nút từ các đối tượng của các lớp nhà cung cấp đặc biệt thực hiện các yêu cầu đó. Điều gì nếu mỗi slot của remote giữ một đối tượng như đối tượng Order của Diner. Sau đó, khi nhấn nút, chúng ta có thể gọi phương thức tương đương của Phương thức "orderUp()" trên đối tượng này và bật đèn mà không cần đến remote để biết chi tiết về cách làm cho đèn sáng hoặc những đối tượng đang tạo ra chúng xảy ra.

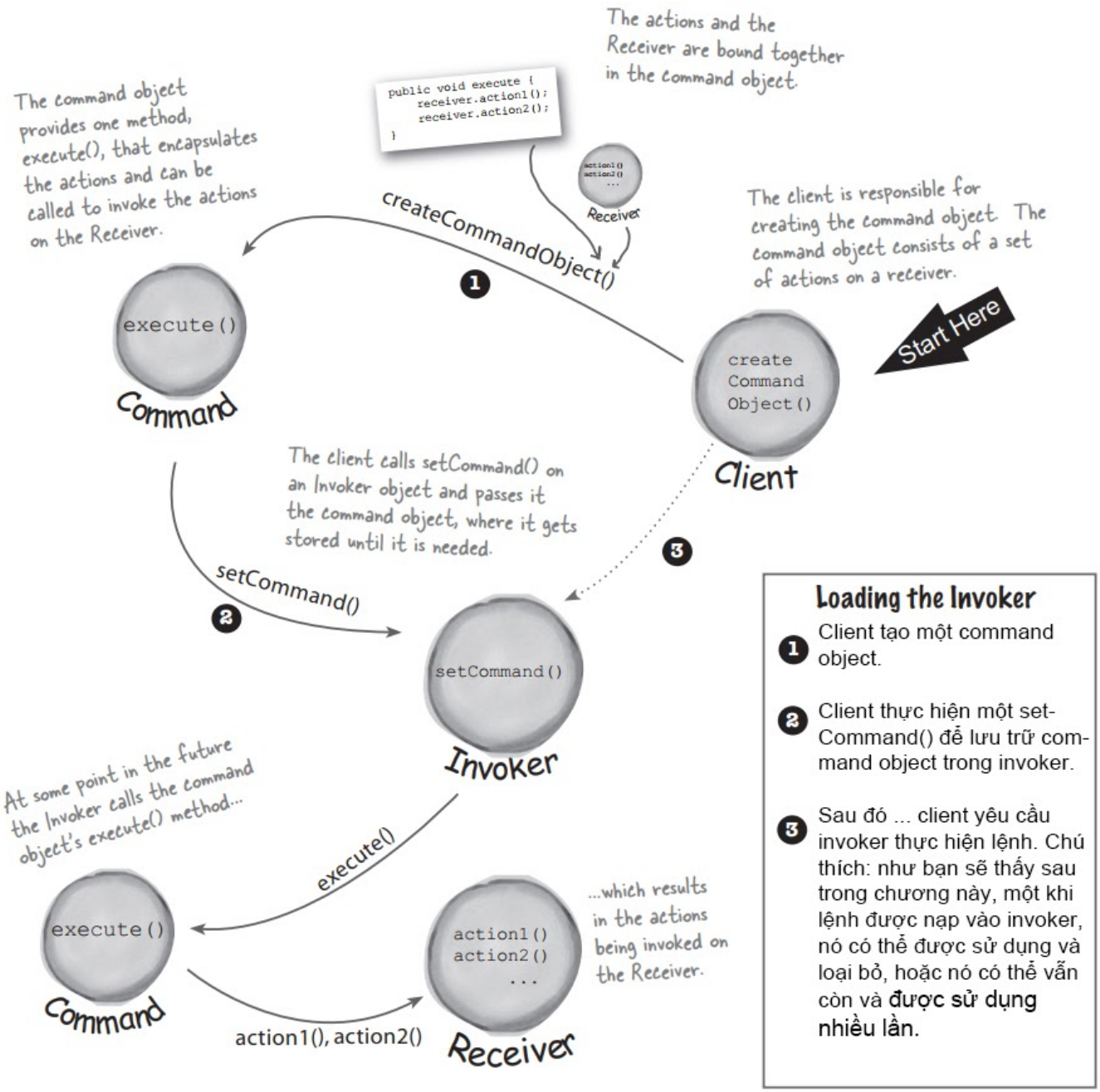
Bây giờ, hãy chuyển đổi một chút và lập bản đồ cho tất cả các bữa tối ở Objectville Diner với Command Pattern…

Sử dụng sức mạnh bộ não

Trước khi chúng ta tiếp tục, hãy dành thời gian nghiên cứu sơ đồ phía trên cùng với vai trò và trách nhiệm của Diner cho đến khi bạn nghĩ rằng bạn đã xử lý được các đối tượng và mối quan hệ của Objectville Diner. Khi bạn đã làm xong việc đó, chúng ta sẽ tiếp tục với Command Pattern!

Từ Objectville Diner đến Command Pattern

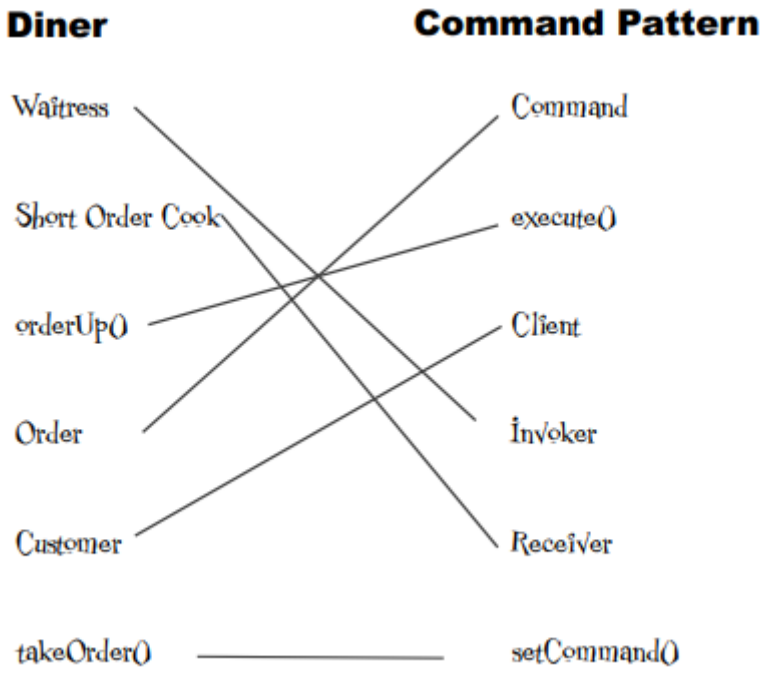
Được rồi, chúng tôi đã dành đủ thời gian ở Objectville Diner để chúng tôi biết tất cả các tính cách và trách nhiệm của chúng. Bây giờ chúng tôi sẽ làm lại sơ đồ của Objectville Diner để phản ánh Mẫu Command. Bạn sẽ thấy rằng tất cả các Object đều giống nhau; chỉ có tên đã thay đổi.



Dùng não của bạn để nối các đối tượng và phương thức Diner với các tên tương ứng của Command Pattern.

Diner	Command Pattern
Waitress	Command
Short Order Cook	execute()
orderUp()	Client
Order	Invoker
Customer	Receiver
takeOrder()	setCommand()

Đáp án:



Command object đầu tiên của chúng ta

Có phải đó là khoảng thời gian chúng ta xây dựng command object đầu tiên không? Hãy đi tiếp và viết code cho remote. Mặc dù chúng ta chưa biết cách thiết kế API điều khiển từ xa, nhưng việc xây dựng một vài thứ từ dưới lên có thể giúp chúng ta…

IMPLEMENTING COMMAND INTERFACE

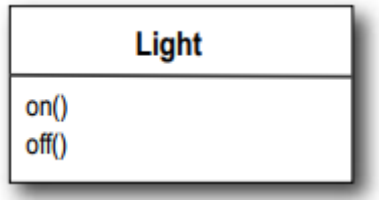
Điều đầu tiên: tất cả các command objects implements cùng một interface, chứa một phương thức. Trong Diner chúng tôi gọi phương thức này là **orderUp()**; tuy nhiên, chúng ta thường chỉ sử dụng tên **execute().**

Đây là giao diện Command:

```
public interface Command {
    public void execute();
}
```

Simple. All we need is one method called execute().

IMPLEMENTING MỘT COMMAND ĐỂ BẬT ĐÈN



Bây giờ, hãy nói rằng bạn muốn implement một command để bật đèn lên. Đề cập đến tập hợp các lớp nhà cung cấp của chúng tôi, lớp Light có hai phương thức: **on()** và **off()**. Đây là cách bạn có thể thực hiện điều này như một command:

This is a command, so we need to implement the Command interface.

```
public class LightOnCommand implements Command {
    Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.on();
    }
}
```

The constructor is passed the specific light that this command is going to control – say the living room light – and stashes it in the light instance variable. When execute gets called, this is the light object that is going to be the Receiver of the request.

The execute method calls the on() method on the receiving object, which is the light we are controlling.

Bây giờ bạn đã có một lớp **LightOnCommand**, hãy để xem liệu chúng ta có thể đưa nó vào sử dụng…

Dùng command object

Được rồi, hãy làm mọi thứ đơn giản: giả sử chúng ta có một remote chỉ với một button và slot tương ứng để giữ một thiết bị để điều khiển:

```
public class SimpleRemoteControl {
    Command slot;

    public SimpleRemoteControl() {}

    public void setCommand(Command command) {
        slot = command;
    }

    public void buttonWasPressed() {
        slot.execute();
    }
}
```

We have one slot to hold our command, which will control one device.

We have a method for setting the command the slot is going to control. This could be called multiple times if the client of this code wanted to change the behavior of the remote button.

This method is called when the button is pressed. All we do is take the current command bound to the slot and call its execute() method.

Tạo một bài test đơn giản để sử dụng Remote Control

Ở đây, chỉ cần một chút code để kiểm tra điều khiển từ xa đơn giản. Chúng ta hãy xem và chúng tôi sẽ chỉ ra chúng tương ứng với phần nào trong Command pattern:

```
public class RemoteControlTest {
    public static void main(String[] args) {
        SimpleRemoteControl remote = new SimpleRemoteControl();
        Light light = new Light();
        LightOnCommand lightOn = new LightOnCommand(light);

        remote.setCommand(lightOn);
        remote.buttonWasPressed();
    }
}
```

This is our Client in Command Pattern-speak.

The remote is our Invoker; it will be passed a command object that can be used to make requests.

Now we create a Light object, this will be the Receiver of the request.

Here, create a command and pass the Receiver to it.

Here, pass the command to the Invoker.

And then we simulate the button being pressed.

Here's the output of running this test code!

```
File Edit Window Help DinerFoodYum
%java RemoteControlTest
Light is On
%
```

Được rồi, đã đến lúc bạn thực hiện lớp **GarageDoorOpenCommand**. Đầu tiên, cung cấp code cho lớp bên dưới. Bạn sẽ cần sơ đồ lớp **GarageDoor**.

```
public class GarageDoorOpenCommand
        implements Command {




}
```

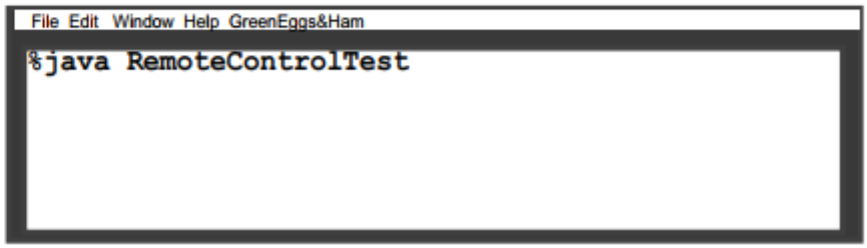
Your code here

Bây giờ bạn đã có lớp của mình, output của đoạn code sau là gì? (Gợi ý: phương thức **up()** của **GarageDoor** in ra "*Garage Door is Open*" khi chạy.)

```
public class RemoteControlTest {
    public static void main(String[] args) {
        SimpleRemoteControl remote = new SimpleRemoteControl();
        Light light = new Light();
        GarageDoor garageDoor = new GarageDoor();
        LightOnCommand lightOn = new LightOnCommand(light);
        GarageDoorOpenCommand garageOpen =
            new GarageDoorOpenCommand(garageDoor);


        remote.setCommand(lightOn);
        remote.buttonWasPressed();
        remote.setCommand(garageOpen);
        remote.buttonWasPressed();
    }
}
```

Your output here.



Đáp án:

```
public class GarageDoorOpenCommand implements Command {
    GarageDoor garageDoor;
    public GarageDoorOpenCommand(GarageDoor garageDoor) {
        this.garageDoor = garageDoor;
    }

    public void execute() {
        garageDoor.up();
    }
}
```

```
File  Edit  Window  Help  GreenEggs&Ham
%java RemoteControlTest

Light is on
Garage Door is Open

%
```

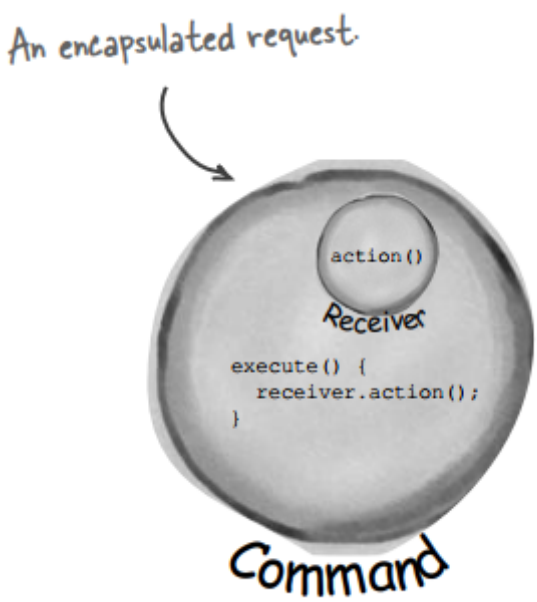
Định nghĩa Command Pattern

Bạn đã có thời gian của mình ở Objectville Diner, bạn đã thực hiện một phần remote control API và trong quá trình đó, bạn đã có một cái nhìn khá tốt về cách các lớp và các đối tượng tương tác trong Command Pattern. Bây giờ chúng tôi sẽ định nghĩa Command Pattern và tìm hiểu tất cả các chi tiết.

Hãy bắt đầu với định nghĩa chính thức của nó:

> **The Command Pattern** encapsulates a request as an object, thereby letting you parameterize other objects with different requests, queue or log requests, and support undoable operations.

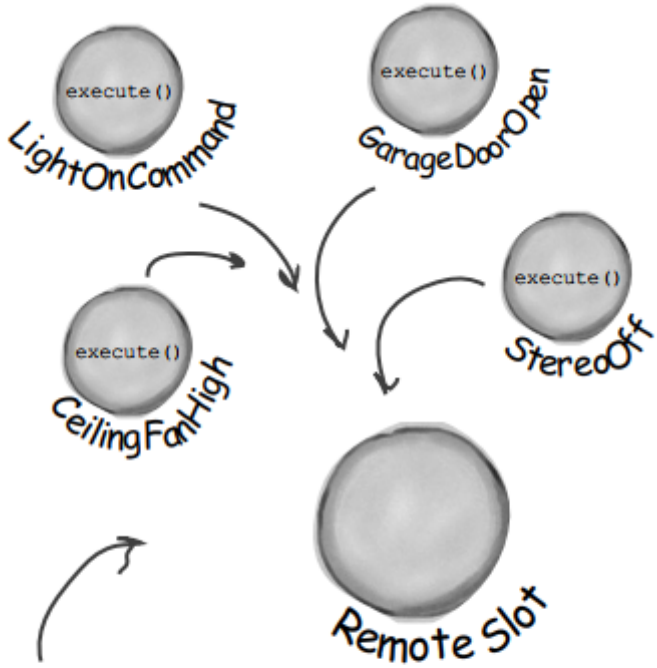
(Command Pattern đóng gói yêu cầu thành đối tượng độc lập, có thể được sử dụng để tham số hóa các đối tượng khác với các yêu cầu khác nhau như log, queue, và hỗ trợ undo.)



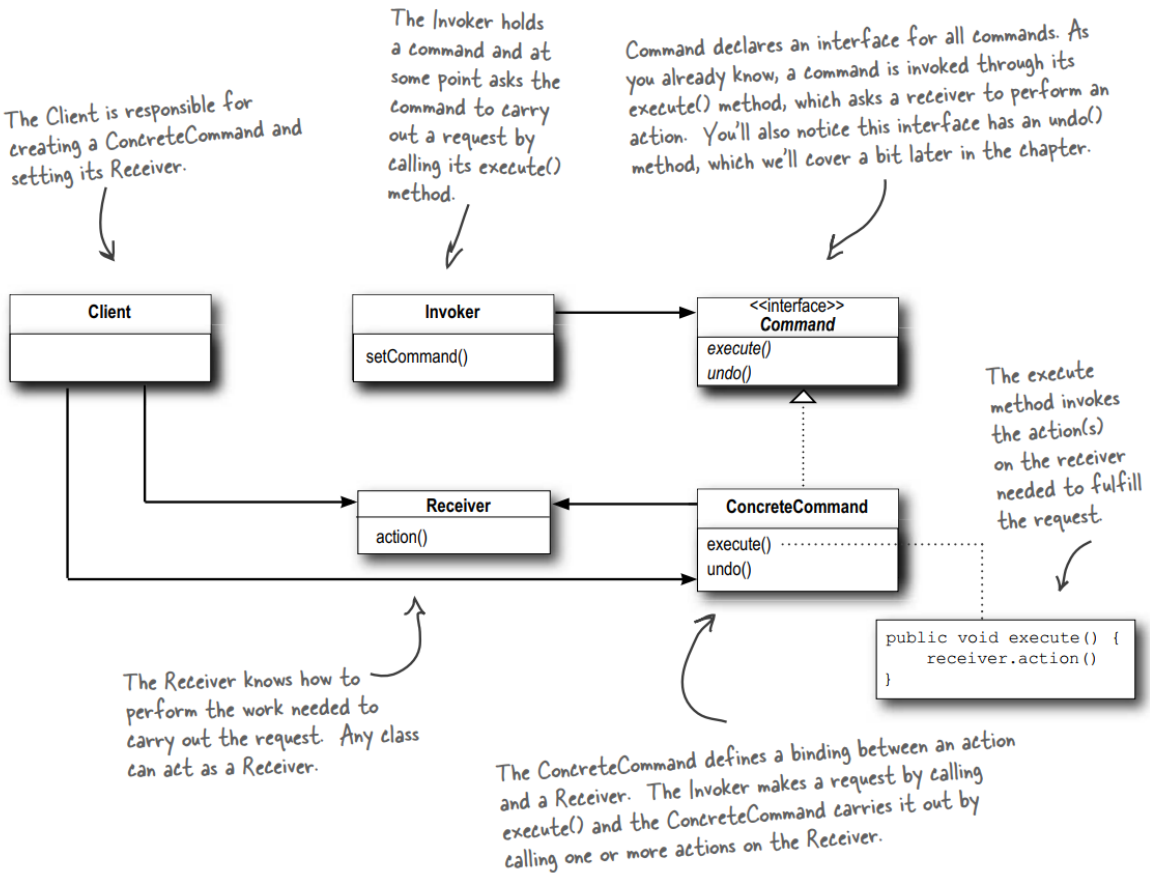
Hãy bước qua điều này. Chúng ta biết rằng một command object đóng gói một yêu cầu bằng cách map chúng với một tập hợp các hành động trên một receiver cụ thể. Để đạt được điều này, nó đóng gói các hành động và receiver thành một đối tượng chỉ một phương thức **execute()**. Khi được gọi, **execute()** sẽ gọi đến phương thức trên receiver để thực hiện hành động. Nhìn từ bên ngoài, không có đối tượng nào khác thực sự biết những hành động nào được thực hiện trên receiver; chúng chỉ biết rằng nếu chúng gọi phương thức **execute()**, request của chúng sẽ được thực hiện.

Chúng ta cũng đã thấy một vài ví dụ về tham số hóa một đối tượng bằng một command. Trở lại quán ăn, Waitress được tham số hóa với nhiều Order trong suốt cả ngày. Trong **SimpleRemoteControl**, trước tiên, chúng tôi đã load vào slot button bằng một "light on" command và sau đó thay thế nó bằng một lệnh "garage door open". Giống như Waitress, remote slot của bạn không quan tâm command object nào được đặt vào, miễn là nó implements giao diện Command.

Những gì chúng ta còn chưa gặp phải là sử dụng các command để cài đặt queues, logs và hỗ trợ undo các hành động. Đừng lo lắng, đó là những phần mở rộng khá đơn giản của Command Pattern cơ bản và chúng ta sẽ sớm nhận được chúng. Chúng ta cũng có thể dễ dàng hỗ trợ những gì mà người ta gọi là Meta Command Pattern khi chúng ta có những điều cơ bản. Meta Command Pattern cho phép bạn tạo các macro command để bạn có thể thực thi nhiều command cùng một lúc.



Định nghĩa Command Pattern: Sơ đồ lớp



Sử dụng sức mạnh bộ não

Làm thế nào để thiết kế Command Pattern hỗ trợ việc tách invoker và receiver?

Sue: Được rồi, tôi nghĩ rằng bây giờ ta đã có một cái nhìn tốt về Command Pattern. Joe thật tuyệt vời, tôi nghĩ chúng ta sẽ trông giống như những siêu sao sau khi hoàn thành API remote.

Mary: Tôi cũng vậy. Vì vậy, chúng ta bắt đầu từ đâu?

Sue: Giống như chúng ta đã làm trong **SimpleRemote**, chúng ta cần cung cấp một cách để gán command cho các slots. Trong trường hợp của chúng ta, chúng ta có 7 slots, mỗi slots có một nút "on" và "off ". Vì vậy, chúng ta có thể gán các command cho remote như thế này:

```
onCommands[0]  = onCommand;
offCommands[0] = offCommand;
```

Mary: Điều đó có ý nghĩa, ngoại trừ các Light objects. Làm thế nào để điều khiển từ xa biết đèn của phòng khách hay đèn của nhà bếp?

Sue: Ah, chỉ vậy à, không thành vấn đề! Điều khiển từ xa không biết gì ngoài cách gọi **execute()** trên command object tương ứng khi nhấn nút.

Mary: Vâng, tôi đã hiểu điều đó, nhưng trong quá trình thực hiện, làm thế nào để chúng ta đảm bảo đúng đối tượng đang bật và tắt đúng thiết bị?

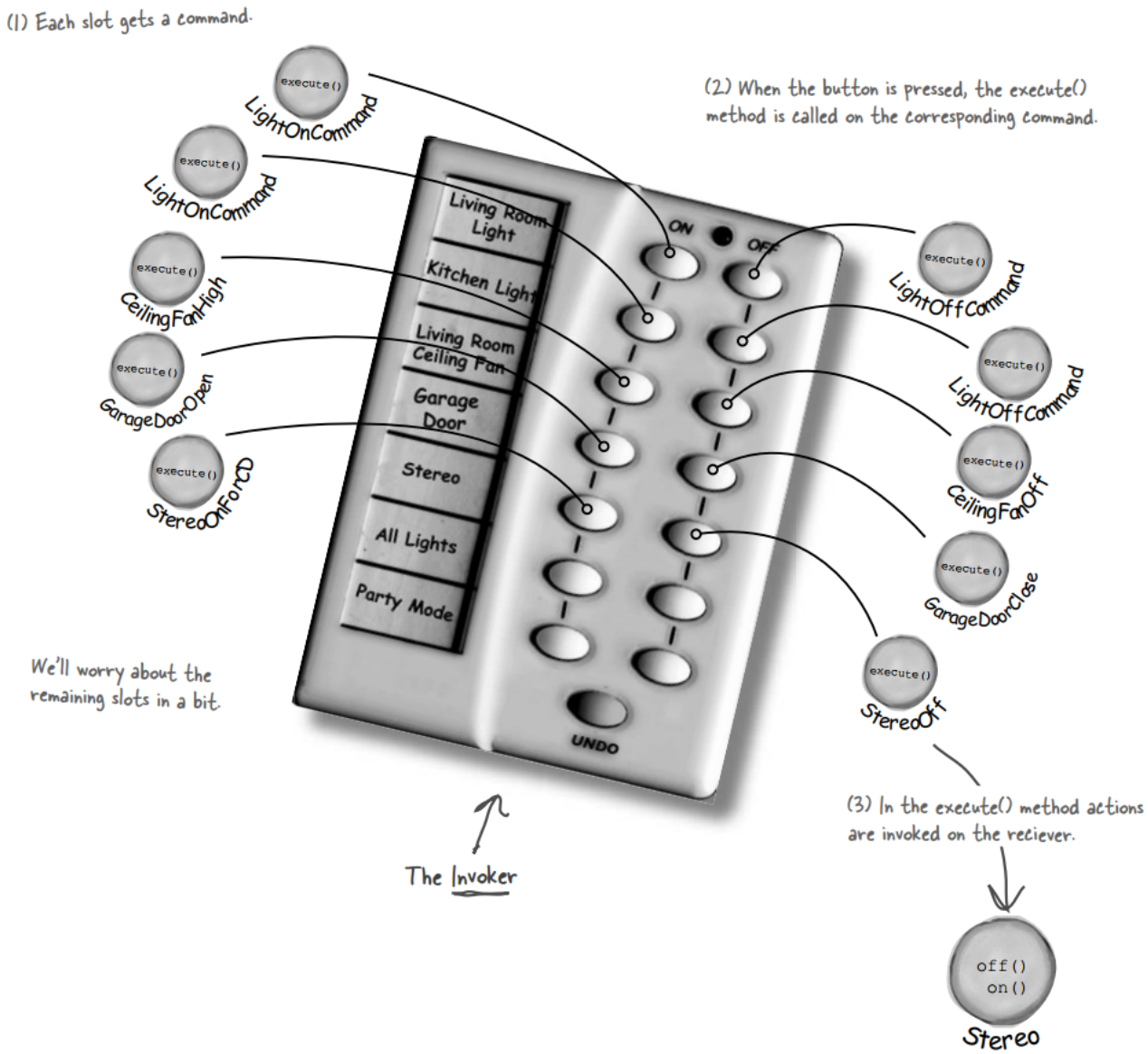
Sue: Khi chúng ta tạo các command được load vào điều khiển từ xa, chúng ta tạo một **LightCommand** được gắn vào đối tượng Living Room Light và một command khác được gắn với đối tượng Kitchen Light. Hãy nhớ rằng, receiver bị ràng buộc với command mà nó đóng gói. Vì vậy, tại thời điểm nhấn nút, không ai quan tâm đến light của phòng nào, điều đúng sẽ xảy ra khi phương thức **execute()** được gọi.

Mary: Tôi nghĩ rằng tôi đã hiểu được nó. Hãy thực hiện chương trình remote và tôi nghĩ điều này sẽ rõ ràng hơn!

Sue: Nghe hay đấy. Bắt đầu thôi…

Gán Command vào slot

Vì vậy, chúng ta có một kế hoạch: Chúng ta sẽ gán mỗi slot của điều khiển từ xa cho một command. Điều này làm cho remote sẽ điều khiển invoker của chúng ta. Khi nhấn nút, phương thức **execute()** sẽ được gọi trên command tương ứng, dẫn đến các hành động sẽ được gọi trên receiver (như đèn, quạt trần, stereos).



Implementing Remote Control

```
public class RemoteControl {
    Command[] onCommands;
    Command[] offCommands;

    public RemoteControl() {
        onCommands = new Command[7];
        offCommands = new Command[7];

        Command noCommand = new NoCommand();
        for (int i = 0; i < 7; i++) {
            onCommands[i] = noCommand;
            offCommands[i] = noCommand;
        }
    }

    public void setCommand(int slot, Command onCommand, Command offCommand) {
        onCommands[slot] = onCommand;
        offCommands[slot] = offCommand;
    }

    public void onButtonWasPushed(int slot) {
        onCommands[slot].execute();
    }

    public void offButtonWasPushed(int slot) {
        offCommands[slot].execute();
    }

    public String toString() {
        StringBuffer stringBuff = new StringBuffer();
        stringBuff.append("\n------ Remote Control -------\n");
        for (int i = 0; i < onCommands.length; i++) {
            stringBuff.append("[slot " + i + "] " + onCommands[i].getClass().getName()
                + "    " + offCommands[i].getClass().getName() + "\n");
        }
        return stringBuff.toString();
    }
}
```

This time around the remote is going to handle seven On and Off commands, which we'll hold in corresponding arrays.

In the constructor all we need to do is instantiate and initialize the on and off arrays.

The setCommand() method takes a slot position and an On and Off command to be stored in that slot. It puts these commands in the on and off arrays for later use.

When an On or Off button is pressed, the hardware takes care of calling the corresponding methods onButtonWasPushed() or offButtonWasPushed().

We've overwritten toString() to print out each slot and its corresponding command. You'll see us use this when we test the remote control.

Cài đặt Command Pattern

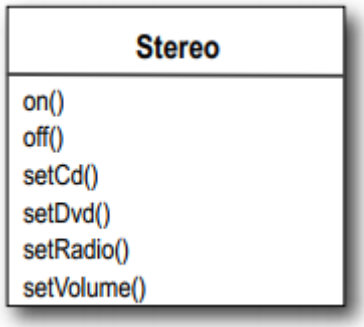
Chà, chúng tôi đã bắt đầu thực hiện **LightOnCommand** cho **SimpleRemoteControl**. Chúng ta có thể đặt code tương tự vào đây và mọi thứ hoạt động. Các command "Off" không khác nhau; trong thực tế, **LightOffCommand** trông như thế này:

```
public class LightOffCommand implements Command {
    Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.off();
    }
}
```

The LightOffCommand works exactly the same way as the LightOnCommand, except that we are binding the receiver to a different action: the off() method.



Hãy thử một cái gì đó thử thách hơn một chút; Làm thế nào về việc viết command on và off cho Stereo? Được rồi, off rất dễ, chúng ta chỉ cần liên kết phương thức Stereo với phương thức **off()** trong **StereoOffCommand**. On thì phức tạp hơn một chút; hãy nói rằng chúng tôi muốn viết một **StereoOnWithCDCommand**…

```
public class StereoOnWithCDCommand implements Command {
    Stereo stereo;

    public StereoOnWithCDCommand(Stereo stereo) {
        this.stereo = stereo;
    }

    public void execute() {
        stereo.on();
        stereo.setCD();
        stereo.setVolume(11);
    }
}
```

Just like the LightOnCommand, we get passed the instance of the stereo we are going to be controlling and we store it in a local instance variable.

To carry out this request, we need to call three methods on the stereo: first, turn it on, then set it to play the CD, and finally set the volume to 11. Why 11? Well, it's better than 10, right?

Không tệ lắm. Hãy xem phần còn lại của các lớp từ nhà cung cấp; bây giờ, bạn chắc chắn có thể "hạ gục" phần còn lại của các lớp Command mà chúng ta cần.

Đặt Remote Control đúng nơi của nó

Công việc của chúng ta với điều khiển từ xa được thực hiện khá nhiều; tất cả những gì chúng ta cần làm là chạy một số thử nghiệm và tập hợp một số tài liệu để mô tả API. Home Automatic hay Bust, Inc. chắc chắn sẽ rất ấn tượng, bạn có nghĩ vậy không? Chúng tôi đã quản lý để đưa ra một thiết kế sẽ cho phép họ sản xuất một cái remote dễ bảo trì và họ sẽ không gặp khó khăn gì trong việc thuyết phục các nhà cung cấp viết một số lớp command đơn giản trong tương lai vì họ rất thích sự dễ viết.

Hãy thử nghiệm code này!

```
public class RemoteLoader {

    public static void main(String[] args) {
        RemoteControl remoteControl = new RemoteControl();

        Light livingRoomLight = new Light("Living Room");
        Light kitchenLight = new Light("Kitchen");
        CeilingFan ceilingFan= new CeilingFan("Living Room");
        GarageDoor garageDoor = new GarageDoor("");
        Stereo stereo = new Stereo("Living Room");

        LightOnCommand livingRoomLightOn =
                new LightOnCommand(livingRoomLight);
        LightOffCommand livingRoomLightOff =
                new LightOffCommand(livingRoomLight);
        LightOnCommand kitchenLightOn =
                new LightOnCommand(kitchenLight);
        LightOffCommand kitchenLightOff =
                new LightOffCommand(kitchenLight);

        CeilingFanOnCommand ceilingFanOn =
                new CeilingFanOnCommand(ceilingFan);
        CeilingFanOffCommand ceilingFanOff =
                new CeilingFanOffCommand(ceilingFan);

        GarageDoorUpCommand garageDoorUp =
                new GarageDoorUpCommand(garageDoor);
        GarageDoorDownCommand garageDoorDown =
                new GarageDoorDownCommand(garageDoor);

        StereoOnWithCDCommand stereoOnWithCD =
                new StereoOnWithCDCommand(stereo);
        StereoOffCommand  stereoOff =
                new StereoOffCommand(stereo);
```

Create all the devices in their proper locations.

Create all the Light Command objects.

Create the On and Off for the ceiling fan.

Create the Up and Down commands for the Garage.

Create the stereo On and Off commands.

```
        remoteControl.setCommand(0, livingRoomLightOn, livingRoomLightOff);
        remoteControl.setCommand(1, kitchenLightOn, kitchenLightOff);
        remoteControl.setCommand(2, ceilingFanOn, ceilingFanOff);
        remoteControl.setCommand(3, stereoOnWithCD, stereoOff);

        System.out.println(remoteControl);

        remoteControl.onButtonWasPushed(0);
        remoteControl.offButtonWasPushed(0);
        remoteControl.onButtonWasPushed(1);
        remoteControl.offButtonWasPushed(1);
        remoteControl.onButtonWasPushed(2);
        remoteControl.offButtonWasPushed(2);
        remoteControl.onButtonWasPushed(3);
        remoteControl.offButtonWasPushed(3);
    }
}
```

Now that we've got all our commands, we can load them into the remote slots.

Here's where we use our toString() method to print each remote slot and the command that it is assigned to.

All right, we are ready to roll! Now, we step through each slot and push its On and Off button.

Now, let's check out the execution of our remote control test...

```
File  Edit  Window  Help  CommandsGetThingsDone

% java RemoteLoader
------ Remote Control -------
[slot 0] headfirst.command.remote.LightOnCommand         headfirst.command.remote.LightOffCommand
[slot 1] headfirst.command.remote.LightOnCommand         headfirst.command.remote.LightOffCommand
[slot 2] headfirst.command.remote.CeilingFanOnCommand    headfirst.command.remote.CeilingFanOffCommand
[slot 3] headfirst.command.remote.StereoOnWithCDCommand  headfirst.command.remote.StereoOffCommand
[slot 4] headfirst.command.remote.NoCommand              headfirst.command.remote.NoCommand
[slot 5] headfirst.command.remote.NoCommand              headfirst.command.remote.NoCommand
[slot 6] headfirst.command.remote.NoCommand              headfirst.command.remote.NoCommand

Living Room light is on
Living Room light is off
Kitchen light is on
Kitchen light is off
Living Room ceiling fan is on high
Living Room ceiling fan is off
Living Room stereo is on
Living Room stereo is set for CD input
Living Room Stereo volume set to 11
Living Room stereo is off

%
```

On slots

Off Slots

Our commands in action! Remember, the output from each device comes from the vendor classes. For instance, when a light object is turned on it prints "Living Room light is on."

Đợi một chút, cái gì là NoCommand được load trong các slot từ 4 đến 6?

Quan sát tốt! Chúng tôi đã lén đặt thêm một chút dòng lệnh. Trong điều khiển từ xa, chúng tôi đã không muốn đặt câu if xem command có được load mỗi lần chúng tôi tham chiếu một slot không. Chẳng hạn, trong phương thức **onButtonWasPushed()**, chúng ta sẽ cần code như thế này:

```
public void onButtonWasPushed(int slot) {
    if (onCommands[slot] != null) {
        onCommands[slot].execute();
    }
}
```

Vì vậy, làm thế nào để chúng ta không cần đặt câu lệnh if? Implements một command mà không cần kiểm tra!

```
public class NoCommand implements Command {
    public void execute() { }
}
```

Sau đó, trong constructor **RemoteControl** của bạn, chúng tôi gán cho mọi slot một đối tượng **NoCommand** theo mặc định và chúng tôi biết rằng chúng tôi sẽ luôn có một command để gọi trong mỗi slot.

```
Command noCommand = new NoCommand();
for (int i = 0; i < 7; i++) {
    onCommands[i] = noCommand;
    offCommands[i] = noCommand;
}
```

Vì vậy, trong output của lần chạy thử của chúng tôi, bạn sẽ thấy vài slot chưa được gán command, ngoài đối tượng **NoCommand** mặc định mà chúng tôi đã gán khi chúng tôi tạo **RemoteControl**.

Đối tượng **NoCommand** là một ví dụ về null object. Một **Null object** rất hữu ích khi bạn không có một đối tượng có ý nghĩa để return và bạn muốn loại bỏ trách nhiệm xử lý kiểm tra null khỏi client. Ví dụ, trong Remote control của chúng tôi, chúng tôi không có một đối tượng có ý nghĩa để gán cho những slot chưa có thiết bị kết nối vào, vì vậy chúng tôi đã cung cấp một đối tượng **NoCommand** hoạt động như một thay thế và không làm gì khi phương thức **execute()** của nó được gọi.

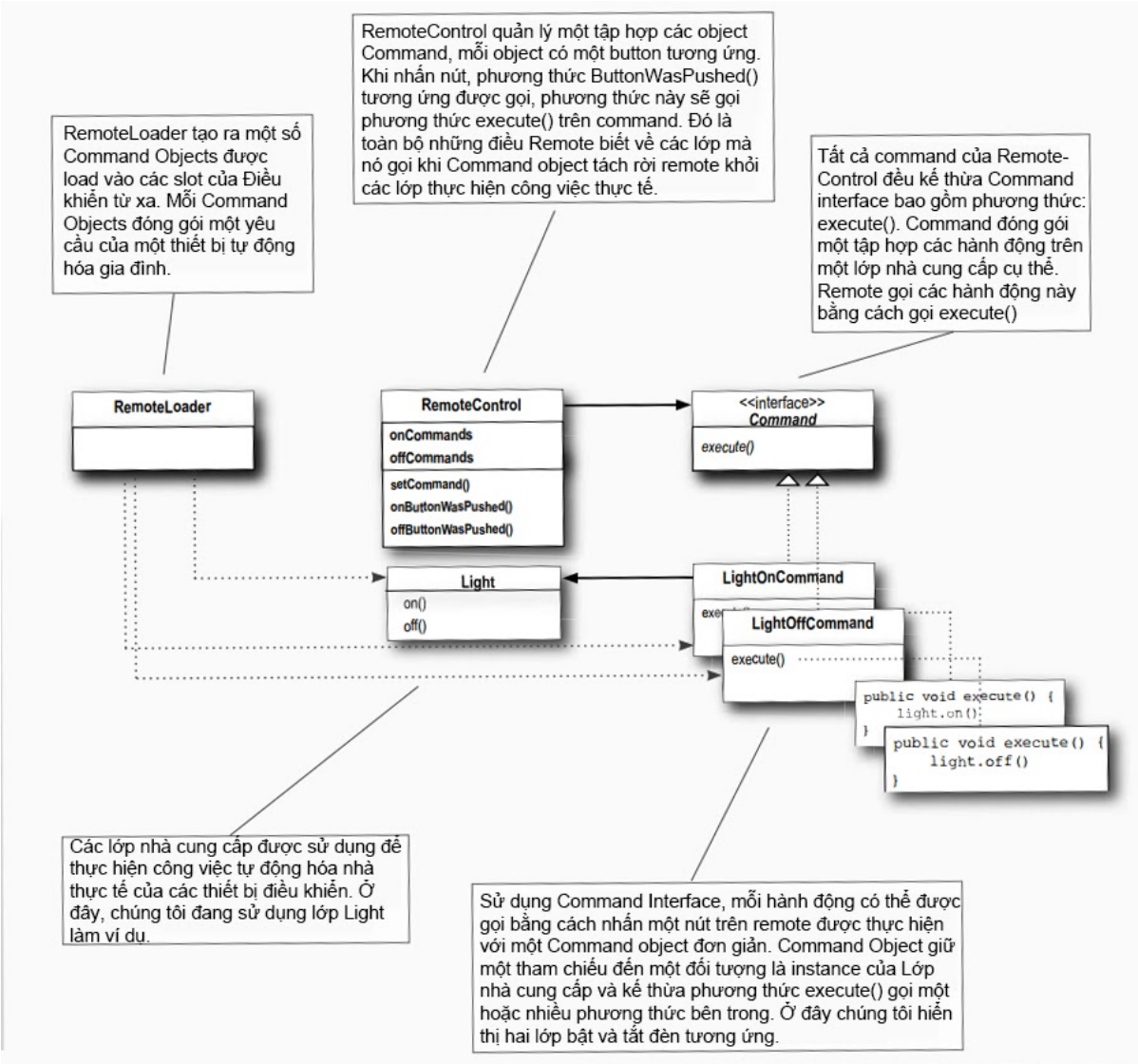
Bạn có thể tìm thấy cách sử dụng cho các **Null Objects** kết hợp với nhiều Mẫu thiết kế và đôi khi bạn thậm chí còn thấy **Null Object** được liệt kê dưới dạng là một Mẫu thiết kế cụ thể.

Đã đến lúc viết tài liệu...

"Thiết kế Remote Control API cho Home Automation or Bust, Inc.,

Chúng tôi hân hạnh giới thiệu với bạn giao diện lập trình ứng dụng và thiết kế sau đây cho Home Automation Remote Control của bạn. Mục tiêu thiết kế chính của chúng tôi là giữ cho code điều khiển từ xa càng đơn giản càng tốt để nó không yêu cầu thay đổi khi các lớp nhà cung cấp mới được sản xuất. Cuối cùng, chúng tôi đã sử dụng Command Pattern để phân tách một cách hợp lý lớp RemoteControl khỏi các lớp nhà cung cấp. Chúng tôi tin rằng điều này sẽ giảm chi phí sản xuất điều khiển từ xa cũng như giảm đáng kể chi phí bảo trì liên tục của bạn.

Sơ đồ lớp sau đây cung cấp tổng quan về thiết kế của chúng tôi:



Làm tốt lắm; Có vẻ như bạn đã đưa ra một thiết kế tuyệt vời, nhưng bạn có quên một điều mà khách hàng yêu cầu không? NÚT UNDO !!!!

Rất tiếc! Chúng tôi gần như đã quên… may mắn thay, một khi chúng tôi có các lớp Command cơ bản, undo rất dễ dàng để thêm vào. Hãy thêm undo các lệnh của chúng tôi và vào điều khiển từ xa…

Chúng ta đang làm gì?

Được rồi, chúng ta cần thêm chức năng để hỗ trợ nút undo trên điều khiển từ xa. Nó hoạt động như thế này: giả sử Đèn phòng khách tắt và bạn nhấn nút bật trên điều khiển từ xa. Rõ ràng là đèn bật sáng. Bây giờ nếu bạn nhấn nút undo thì hành động cuối cùng sẽ bị đảo ngược – trong trường hợp này đèn sẽ tắt. Trước khi chúng ta đi vào các ví dụ phức tạp hơn, hãy để đối tượng Đèn hoạt động với nút undo:

1. Khi các lệnh hỗ trợ undo, chúng có phương thức **undo()** phản chiếu phương thức **execute()**. Bất cứ điều gì **execute()** cuối cùng đã làm, **undo()** sẽ đảo ngược chúng. Vì vậy, trước khi chúng ta có thể thêm chức năng undo vào các command của mình, chúng ta cần thêm một phương thức **undo()** vào giao diện **Command**:

```
public interface Command {
    public void execute();
    public void undo();
}
```

Here's the new undo() method.

Điều đó đã đủ đơn giản. Bây giờ, hãy đi vào Light command và thực hiện phương thức **undo()**.

2. Hãy bắt đầu với **LightOnCommand**: nếu phương thức ***execute()*** của **LightOnCommand** được gọi, thì phương thức ***on()*** được gọi lần cuối. Chúng ta biết rằng ***undo()*** cần phải làm ngược lại điều này bằng cách gọi phương thức ***off()***.

```
public class LightOnCommand implements Command {
    Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.on();
    }

    public void undo() {
        light.off();
    }
}
```

execute() turns the light on, so undo() simply turns the light back off.

Bây giờ cho **LightOffCommand**. Ở đây, phương thức ***undo()*** chỉ cần gọi phương thức ***on()*** của object **Light**.

```
public class LightOffCommand implements Command {
    Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.off();
    }

    public void undo() {
        light.on();
    }
}
```

And here, undo() turns the light back on!

Điều này có thể dễ dàng hơn không? Được rồi, chúng ta chưa hoàn thành; chúng ta cần hỗ trợ một chút vào Điều khiển từ xa để xử lý việc theo dõi nút cuối cùng được nhấn và nút undo được nhấn.

3. Để thêm hỗ trợ cho nút undo, chúng ta chỉ phải thực hiện một vài thay đổi nhỏ cho lớp Remote Control. Ở đây, cách thức mà chúng tôi sẽ thực hiện nó: chúng tôi sẽ thêm một biến đối tượng mới để theo dõi command cuối cùng được gọi; sau đó, bất cứ khi nào nhấn nút undo, chúng tôi sẽ truy xuất command đó và gọi phương thức ***undo()*** của nó.

```
public class RemoteControlWithUndo {
    Command[] onCommands;
    Command[] offCommands;
    Command undoCommand;

    public RemoteControlWithUndo() {
        onCommands = new Command[7];
        offCommands = new Command[7];

        Command noCommand = new NoCommand();
        for(int i=0;i<7;i++) {
            onCommands[i] = noCommand;
            offCommands[i] = noCommand;
        }
        undoCommand = noCommand;
    }

    public void setCommand(int slot, Command onCommand, Command offCommand) {
        onCommands[slot] = onCommand;
        offCommands[slot] = offCommand;
    }

    public void onButtonWasPushed(int slot) {
        onCommands[slot].execute();
        undoCommand = onCommands[slot];
    }

    public void offButtonWasPushed(int slot) {
        offCommands[slot].execute();
        undoCommand = offCommands[slot];
    }

    public void undoButtonWasPushed() {
        undoCommand.undo();
    }

    public String toString() {
        // toString code here...
    }
}
```

This is where we'll stash the last command executed for the undo button.

Just like the other slots, undo starts off with a NoCommand, so pressing undo before any other button won't do anything at all.

When a button is pressed, we take the command and first execute it; then we save a reference to it in the undoCommand instance variable. We do this for both "on" commands and "off" commands.

When the undo button is pressed, we invoke the undo() method of the command stored in undoCommand. This reverses the operation of the last command executed.

Ứng dụng Command Pattern: nút Undo!

Được rồi, hãy test lại một chút để kiểm tra nút undo:

```
public static void main(String[] args) {
    RemoteControlWithUndo remoteControl = new RemoteControlWithUndo();

    Light livingRoomLight = new Light("Living Room");

    LightOnCommand livingRoomLightOn =
            new LightOnCommand(livingRoomLight);
    LightOffCommand livingRoomLightOff =
            new LightOffCommand(livingRoomLight);

    remoteControl.setCommand(0, livingRoomLightOn, livingRoomLightOff);

    remoteControl.onButtonWasPushed(0);
    remoteControl.offButtonWasPushed(0);
    System.out.println(remoteControl);
    remoteControl.undoButtonWasPushed();
    remoteControl.offButtonWasPushed(0);
    remoteControl.onButtonWasPushed(0);
    System.out.println(remoteControl);
    remoteControl.undoButtonWasPushed();
}
```

Create a Light, and our new undo() enabled Light On and Off Commands.

Add the light Commands to the remote in slot 0.

Turn the light on, then off and then undo.

Then, turn the light off, back on and undo.

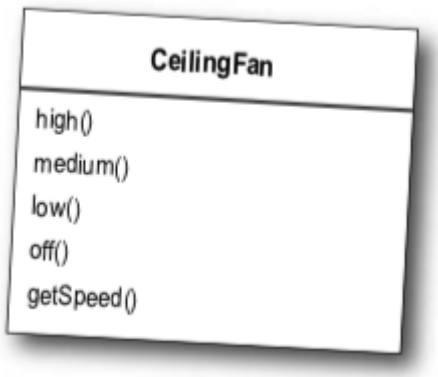
Và đây, kết quả kiểm tra…



Sử dụng state để implement Undo

Được rồi, thực hiện Undo trên lớp Light là hướng dẫn nhưng hơi quá dễ dàng. Thông thường, chúng ta cần quản lý một chút trạng thái (state) để thực hiện undo. Hãy thử một cái gì đó thú vị hơn một chút, như lớp Quạt trần (CeilingFan) từ các lớp nhà cung cấp. Quạt trần cho phép một số tốc độ quạt (speed) được đặt cùng với phương thức off.

Đây là code của **CeilingFan**:



```
public class CeilingFan {
    public static final int HIGH = 3;
    public static final int MEDIUM = 2;
    public static final int LOW = 1;
    public static final int OFF = 0;
    String location;
    int speed;

    public CeilingFan(String location) {
        this.location = location;
        speed = OFF;
    }

    public void high() {
        speed = HIGH;
        // code to set fan to high
    }

    public void medium() {
        speed = MEDIUM;
        // code to set fan to medium
    }

    public void low() {
        speed = LOW;
        // code to set fan to low
    }

    public void off() {
        speed = OFF;
        // code to turn fan off
    }

    public int getSpeed() {
        return speed;
    }
}
```

Notice that the CeilingFan class holds local state representing the speed of the ceiling fan.

These methods set the speed of the ceiling fan.

We can get the current speed of the ceiling fan using getSpeed().



Thêm Undo vào CeilingFan Command

Bây giờ hãy giải quyết việc thêm undo vào các command **CeilingFan** khác nhau. Để làm như vậy, chúng ta cần theo dõi cài đặt tốc độ cuối cùng của quạt và, nếu phương thức **undo()** được gọi, hãy khôi phục quạt về tốc độ trước khi tắt. Đây là code cho lớp **CeilingFanHighCommand**:

```
public class CeilingFanHighCommand implements Command {
    CeilingFan ceilingFan;
    int prevSpeed;

    public CeilingFanHighCommand(CeilingFan ceilingFan) {
        this.ceilingFan = ceilingFan;
    }

    public void execute() {
        prevSpeed = ceilingFan.getSpeed();
        ceilingFan.high();
    }

    public void undo() {
        if (prevSpeed == CeilingFan.HIGH) {
            ceilingFan.high();
        } else if (prevSpeed == CeilingFan.MEDIUM) {
            ceilingFan.medium();
        } else if (prevSpeed == CeilingFan.LOW) {
            ceilingFan.low();
        } else if (prevSpeed == CeilingFan.OFF) {
            ceilingFan.off();
        }
    }
}
```

We've added local state to keep track of the previous speed of the fan.

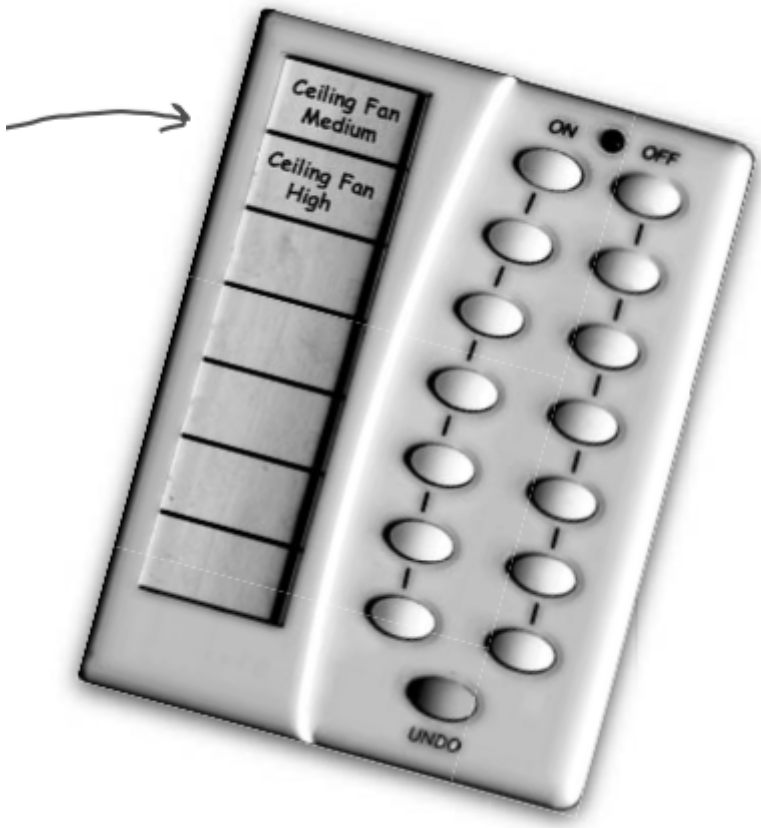
In execute, before we change the speed of the fan, we need to first record its previous state, just in case we need to undo our actions.

To undo, we set the speed of the fan back to its previous speed.

Sử dụng sức mạnh bộ não bạn

Chúng tôi đã có thêm ba **CellingFan** command: *low*, *medium* và *off* làm thế nào để tất cả chúng được thực hiện?

Hãy sẵn sàng để test CellingFan



Thời gian để load lên điều khiển từ xa của chúng ta với các **CellingFan** command. Chúng tôi sẽ load slot 0 cho quạt với tốc độ *medium* và slot 1 với mức *high*. Cả hai nút off tương ứng sẽ giữ lệnh tắt quạt trần.

Đây là kịch bản thử nghiệm của chúng tôi:

```
public class RemoteLoader {

    public static void main(String[] args) {
        RemoteControlWithUndo remoteControl = new RemoteControlWithUndo();

        CeilingFan ceilingFan = new CeilingFan("Living Room");

        CeilingFanMediumCommand ceilingFanMedium =
                new CeilingFanMediumCommand(ceilingFan);
        CeilingFanHighCommand ceilingFanHigh =
                new CeilingFanHighCommand(ceilingFan);
        CeilingFanOffCommand ceilingFanOff =
                new CeilingFanOffCommand(ceilingFan);

        remoteControl.setCommand(0, ceilingFanMedium, ceilingFanOff);
        remoteControl.setCommand(1, ceilingFanHigh, ceilingFanOff);

        remoteControl.onButtonWasPushed(0);
        remoteControl.offButtonWasPushed(0);
        System.out.println(remoteControl);
        remoteControl.undoButtonWasPushed();

        remoteControl.onButtonWasPushed(1);
        System.out.println(remoteControl);
        remoteControl.undoButtonWasPushed();
    }
}
```

Here we instantiate three commands: high, medium, and off.

Here we put medium in slot zero, and high in slot one. We also load up the off commands.

First, turn the fan on medium.

Then turn it off.

Undo! It should go back to medium...

Turn it on to high this time.

And, one more undo; it should go back to medium.

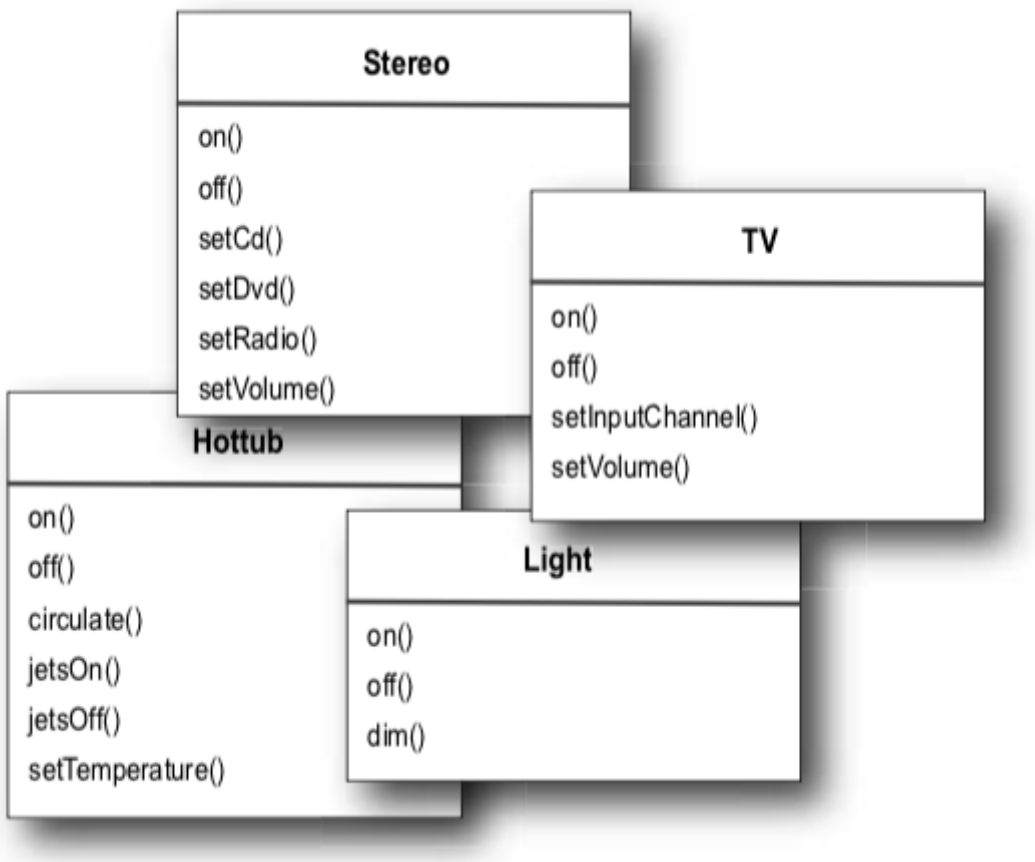
Test lớp Celling Fan của chúng ta

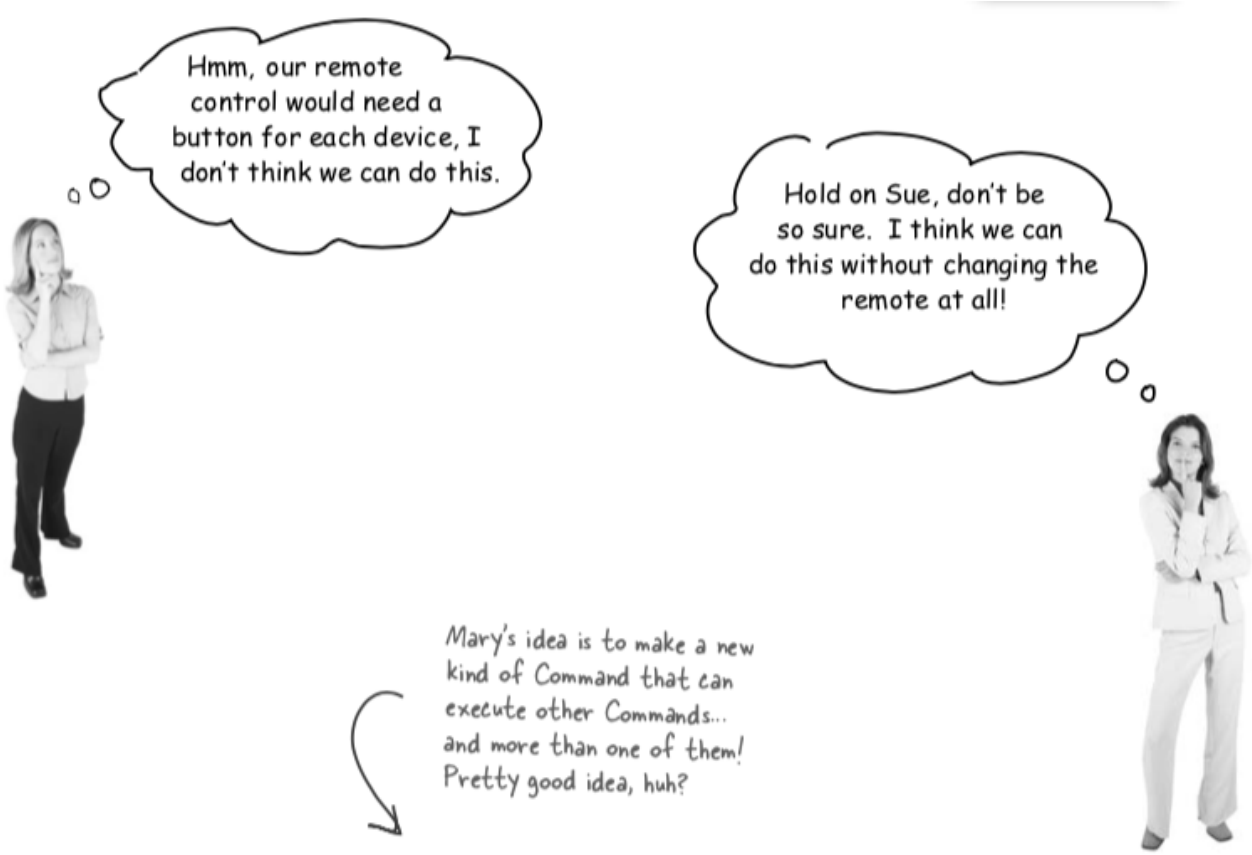
Được rồi, hãy khởi động remote, load nó bằng các command và nhấn một số nút!



Mọi điều khiển từ xa đều cần một Chế độ Party!

Điều gì nếu bạn chỉ cần nhấn một nút và đèn sẽ mờ, stereo và TV được bật và play DVD, bồn nước nóng được bật lên?






```
public class MacroCommand implements Command {
    Command[] commands;

    public MacroCommand(Command[] commands) {
        this.commands = commands;
    }

    public void execute() {
        for (int i = 0; i < commands.length; i++) {
            commands[i].execute();
        }
    }
}
```

Take an array of Commands and store them in the MacroCommand.

When the macro gets executed by the remote, execute those commands one at a time.

Ứng dụng Command Pattern: Macro command

Hãy xem qua cách chúng tôi sử macro command:

1. Đầu tiên chúng ta tạo tập hợp các command mà chúng ta muốn đưa vào macro:

```
Light light = new Light("Living Room");
TV tv = new TV("Living Room");
Stereo stereo = new Stereo("Living Room");
Hottub hottub = new Hottub();

LightOnCommand lightOn = new LightOnCommand(light);
StereoOnCommand stereoOn = new StereoOnCommand(stereo);
TVOnCommand tvOn = new TVOnCommand(tv);
HottubOnCommand hottubOn = new HottubOnCommand(hottub);
```

Create all the devices, a light, tv, stereo, and hot tub.

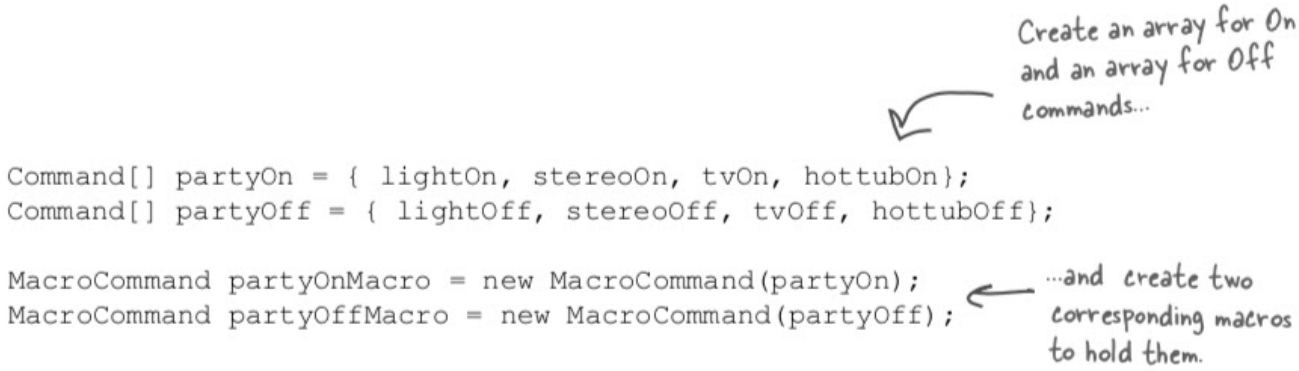
Now create all the On commands to control them.

Đến lượt bạn: Chúng ta cũng sẽ cần các command cho các nút off, viết code để tạo các nút ở đây

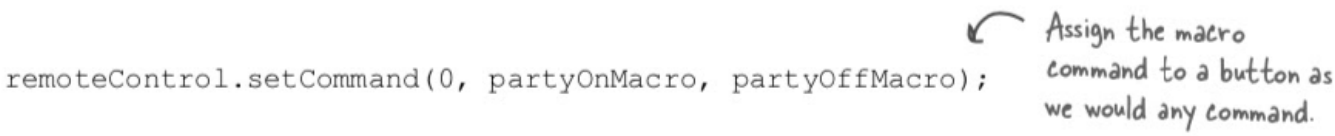
Đáp án:

```
LightOffCommand lightOff = new LightOffCommand(light);
StereoOffCommand stereoOff = new StereoOffCommand(stereo);
TVOffCommand tvOff = new TVOffCommand(tv);
HottubOffCommand hottubOff = new HottubOffCommand(hottub);
```

2. Tiếp theo, chúng ta tạo hai mảng, một cho các lệnh **On** và một cho **Off** và load chúng với các command tương ứng:



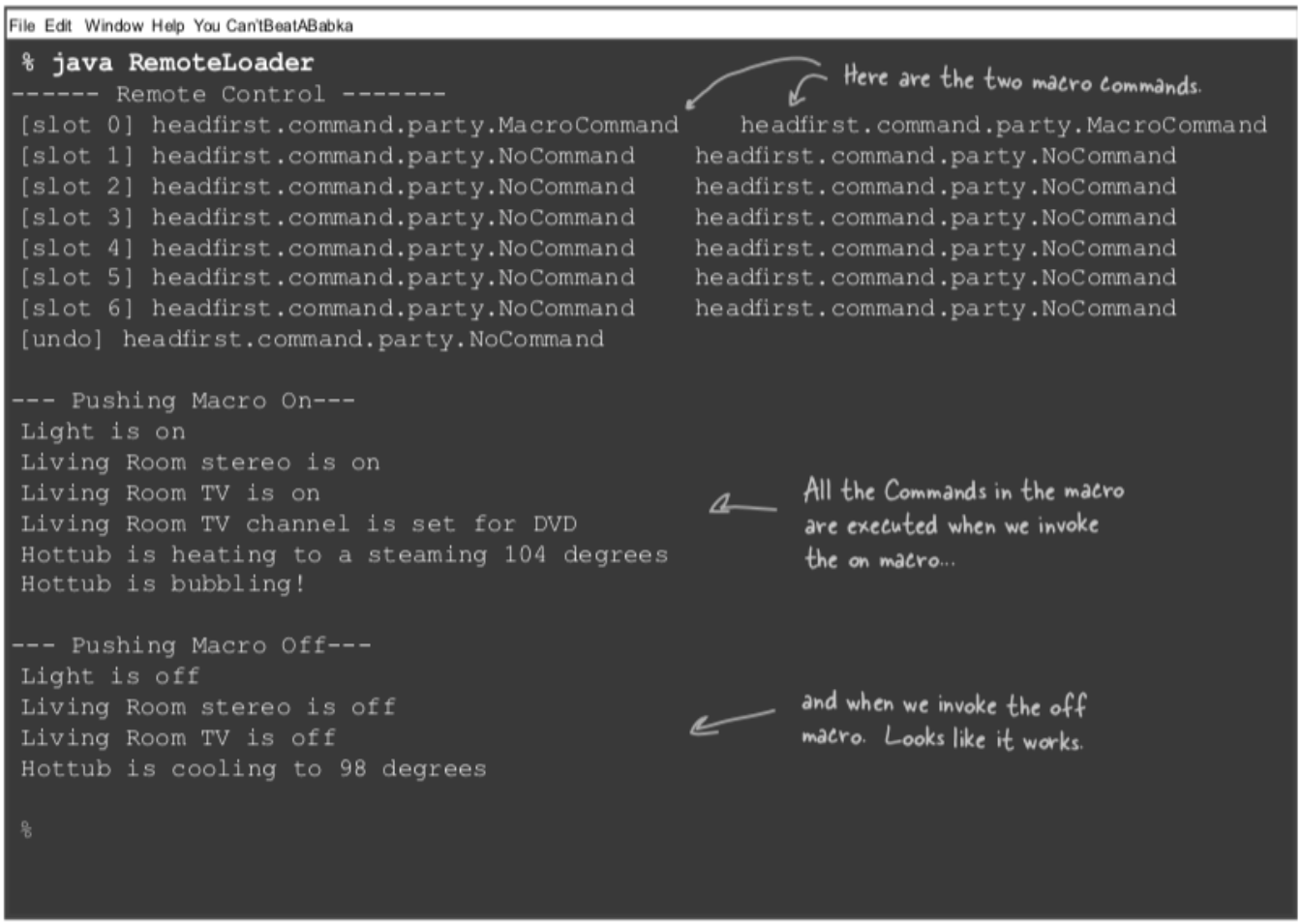
3. Sau đó, chúng ta gán **MacroCommand** cho một button như chúng ta đã làm:



4. Cuối cùng, chúng ta chỉ cần nhấn một số nút và xem điều này hoạt động.

```
System.out.println(remoteControl);
System.out.println("--- Pushing Macro On---");
remoteControl.onButtonWasPushed(0);
System.out.println("--- Pushing Macro Off---");
remoteControl.offButtonWasPushed(0);
```

Here's the output



Bài tập: Điều duy nhất MacroCommand của chúng ta thiếu là chức năng undo. Khi nhấn nút undo sau một macro command, tất cả các command được gọi trong macro phải undo các hành động trước đó của chúng. Đây là code cho MacroCommand; tiếp tục và thực hiện phương thức undo():

```
public class MacroCommand implements Command {
    Command[] commands;

    public MacroCommand(Command[] commands) {
        this.commands = commands;
    }

    public void execute() {
        for (int i = 0; i < commands.length; i++) {
            commands[i].execute();
        }
    }

    public void undo() {
```

```
    }
}
```

Đáp án:

```
public class MacroCommand implements Command {
    Command[] commands;
    public MacroCommand(Command[] commands) {
        this.commands = commands;
    }

    public void execute() {
        for (int i = 0; i < commands.length; i++) {
            commands[i].execute();
        }
    }

    public void undo() {
        for (int i = 0; i < commands.length; i++) {
            commands[i].undo();
        }
    }
}
```

Không có câu hỏi ngớ ngẩn

Hỏi: Tôi luôn cần một receiver phải không? Tại sao command object không thể cài đặt phần chi tiết của ***execute()*** method?

Trả lời: Nói chung, chúng tôi cố gắng cho các command objects, chỉ cần implement một hành động trên receiver; tuy nhiên, có rất nhiều ví dụ về các đối tượng command "thông minh" thực hiện hầu hết, nếu không phải là tất cả, logic cần thiết để thực hiện một yêu cầu. Chắc chắn bạn có thể làm điều này; chỉ cần lưu ý rằng bạn sẽ không còn mức độ tách rời giữa invoker và receiver, bạn cũng không thể tham số hóa các command của bạn với receiver.

Hỏi: Làm thế nào tôi có thể thực hiện một lịch sử hoạt động undo? Nói cách khác, tôi muốn có thể nhấn nút undo nhiều lần.

Trả lời: Câu hỏi tuyệt vời! Nó thực sự rất dễ dàng; thay vì chỉ giữ một tham chiếu đến Command cuối cùng được thực thi, bạn giữ một ngăn xếp Stack các command trước đó. Sau đó, bất cứ khi nào undo được nhấn, invoker của bạn sẽ Pop (lấy) command đầu tiên ra khỏi stack và gọi phương thức **undo()** của command đó.

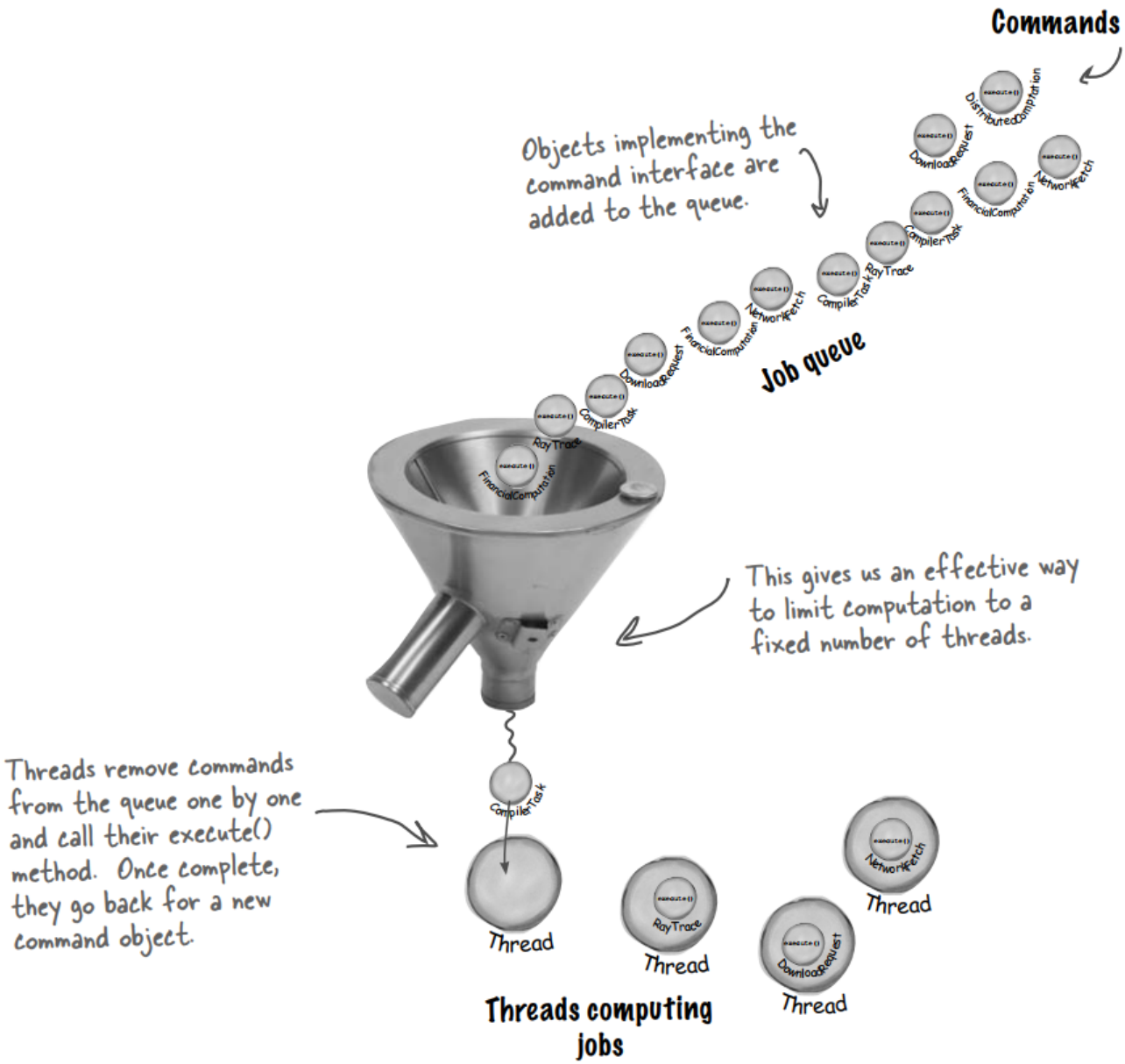
Hỏi: Tôi có thể thực hiện "Party Mode" như một command bằng cách tạo **PartyCommand** và đặt lệnh gọi đến các phương thức **execute()** của Command khác trong phương thức **execute()** của **PartyCommand** không?

Trả lời: Bạn có thể; tuy nhiên, chúng sẽ trở thành "code cứng" trong **PartyCommand**. Đó là một rắc rối lớn? Với **MacroCommand**, bạn có thể quyết định các **Command** nào bạn muốn vào **PartyCommand**, do đó bạn có thể linh hoạt hơn khi sử dụng **MacroCommand**. Nhìn chung, **MacroCommand** là một giải pháp an toàn hơn và yêu cầu ít code mới hơn.

Ứng dụng Command Pattern: Hàng đợi các request

Command cho chúng ta một cách để đóng gói một phần xử lý (một receiver và một tập hợp các hành động) và chuyển nó đi như một đối tượng first-class. Bây giờ, việc xử lí có thể được invoked sau đó khi một số ứng dụng client tạo đối Command. Trong thực tế, nó thậm chí có thể được gọi bởi một thread khác. Chúng ta có thể lấy kịch bản này và áp dụng nó cho nhiều ứng dụng hữu ích như schedulers (đặt lịch chạy), thread pools và job queues.

Tưởng tượng một hàng đợi công việc: bạn thêm các command vào hàng đợi ở một đầu và ở đầu kia là một nhóm các thread. Các thread sẽ chạy đoạn script sau: loại bỏ một command khỏi hàng đợi, gọi phương thức **execute()** của nó, đợi cuộc gọi kết thúc, sau đó loại bỏ đối tượng command đó và truy xuất một command mới.



Lưu ý rằng các lớp hàng đợi công việc được tách rời hoàn toàn khỏi các đối tượng đang thực hiện xử lí. Một phút một Subject có thể tính toán một tính toán, và phút tiếp theo nó có thể lấy một cái gì đó từ network… Các đối tượng hàng đợi công việc không quan tâm; chúng chỉ lấy các command và gọi **execute().** Tương tự, miễn là bạn đặt các đối tượng vào hàng đợi thực hiện Command Pattern, phương thức **execute()** của bạn sẽ được gọi khi có một thread.

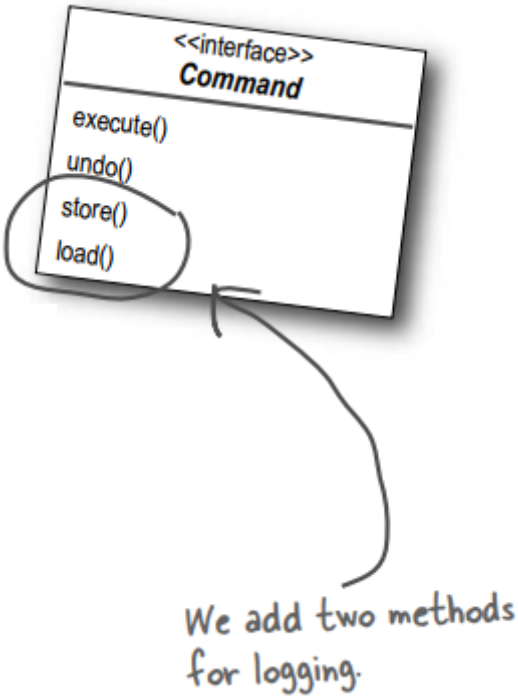
Sử dụng sức mạnh bộ não

Làm thế nào một web server có thể tận dụng dụng một hàng đợi như vậy? Bạn có thể nghĩ đến một ứng dụng nào khác không?

Ứng dụng Command Pattern: logging request

Ngữ nghĩa của một số ứng dụng yêu cầu chúng tôi log tất cả các hành động và có thể phục hồi sau khi gặp sự cố bằng cách khôi phục các hành động đó. Mẫu Command có thể hỗ trợ các ngữ nghĩa này với việc bổ sung hai phương thức: **store()** và **load()**. Trong Java, chúng ta có thể sử dụng object serialization

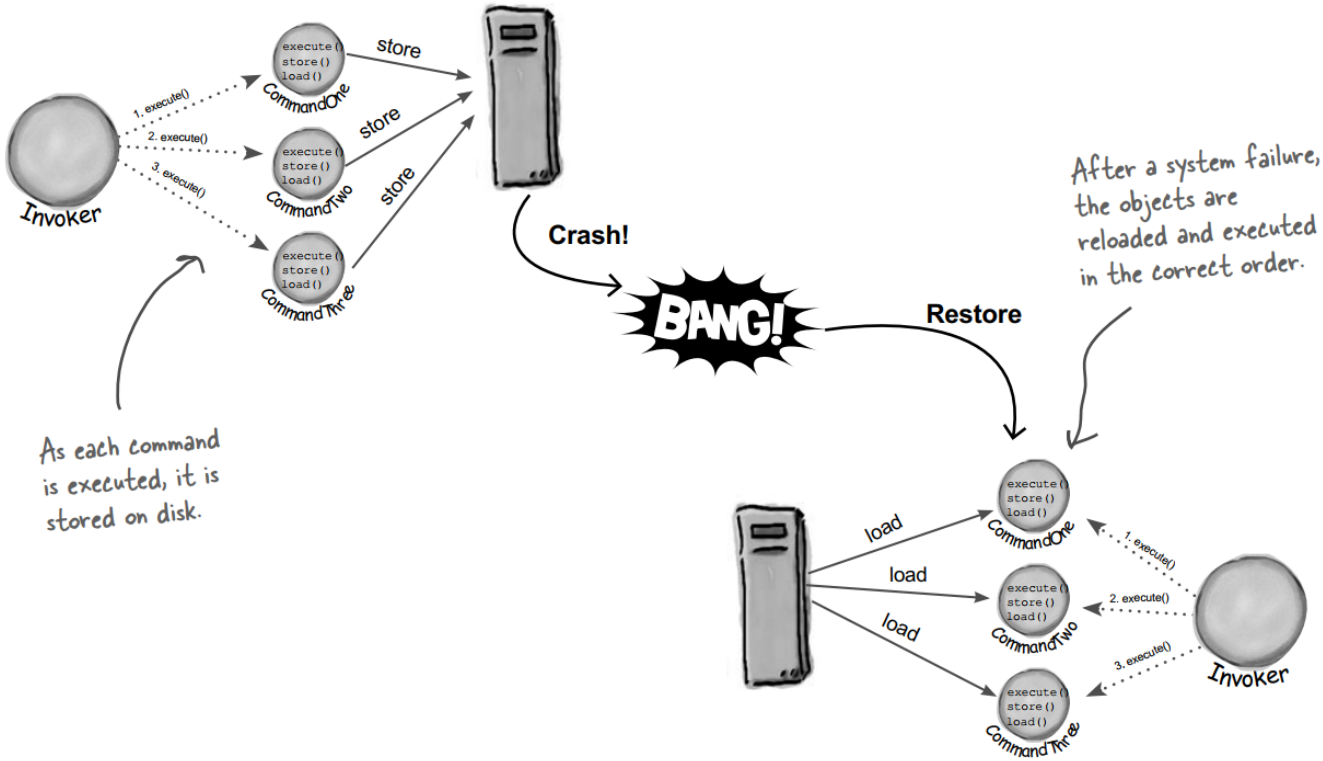
(tuần tự hóa) để thực hiện các phương thức này, nhưng hãy cẩn thận khi sử dụng serialization để □bảo trì.



Cái này hoạt động ra sao? Khi chúng tôi thực thi các command, chúng tôi lưu trữ (store) một lịch sử của chúng trên đĩa. Khi xảy ra sự cố, chúng ta load lại các đối tượng command và gọi các phương thức **execute()** của chúng theo nhóm và theo thứ tự.

Bây giờ, loại log này sẽ không có ý nghĩa đối với một remote control; tuy nhiên, có nhiều ứng dụng gọi các hành động trên các cấu trúc dữ liệu lớn có thể được lưu nhanh chóng mỗi khi có thay đổi. Bằng cách sử dụng log, chúng tôi có thể lưu tất cả các hoạt động kể từ thời điểm kiểm tra cuối cùng (last check point) và nếu có lỗi hệ thống, hãy áp dụng các hoạt động đó cho checkpoint của chúng tôi.

Lấy ví dụ, một ứng dụng bảng tính (spreadsheet, như excel): chúng tôi có thể muốn thực hiện khôi phục lỗi của mình bằng cách ghi log các hành động trên bảng tính thay vì viết một bản sao của bảng tính vào đĩa mỗi khi có thay đổi. Trong các ứng dụng nâng cao hơn, các kỹ thuật này có thể được mở rộng để áp dụng cho các tập hợp hoạt động theo transactional để tất cả các hoạt động sẽ hoặc hoàn thành hoặc không có thao tác nào thực hiện (tìm hiểu transaction database (https://vi.wikipedia.org/wiki/Giao_d%E1%BB%8Bch_c%C6%A1_s%E1%BB%9F_d%E1%BB%AF_li%E1%BB%87u)).



Tóm tắt

- Command Pattern tách riêng một đối tượng, và đưa ra yêu cầu tới đối tượng biết cách thực hiện nó.
- Một đối tượng Command nằm ở trung tâm của việc tách rời này và đóng gói một receiver bằng một hành động (hoặc tập hợp các hành động).
- Một invoker đưa ra yêu cầu của một đối tượng Command bằng cách gọi phương thức execute() của nó, gọi các hành động đó trên receiver.
- Invoker có thể được tham số hóa bằng các Command, thậm chí là gán động khi chạy.
- Command có thể hỗ trợ undo bằng cách thực hiện một phương thức undo khôi phục đối tượng về trạng thái trước đó trước khi phương thức execute() được gọi lần cuối.
- Macro Commands là một phần mở rộng đơn giản của Command cho phép nhiều command được gọi. Tương tự, Macro Commands có thể dễ dàng hỗ trợ undo().
- Trong thực tế, không có gì lạ khi các đối tượng "smart" Command có thể tự thực hiện yêu cầu thay vì ủy thác cho receiver.
- Command cũng có thể được sử dụng để thực hiện các hệ thống *logging* và *transactional*.